

AgenticScholar: Agentic Data Management with Pipeline Orchestration for Scholarly Corpora

HAI LAN, School of Electrical Engineering and Computer Science, The University of Queensland, Australia
TINGTING WANG, School of Electrical Engineering and Computer Science, The University of Queensland, Australia

ZHIFENG BAO, School of Electrical Engineering and Computer Science, The University of Queensland, Australia

GUOLIANG LI, Department of Computer Science and Technology, Tsinghua University, China

DAOMIN JI, School of Electrical Engineering and Computer Science, The University of Queensland, Australia

GE LEE, School of Electrical Engineering and Computer Science, The University of Queensland, Australia

FENG LUO, School of Electrical Engineering and Computer Science, The University of Queensland, Australia

ZI HUANG, School of Electrical Engineering and Computer Science, The University of Queensland, Australia

HAILANG QIU, School of Computer Science, Wuhan University, China

GANG HUA, School of Computer Science and Artificial Intelligence, Wuhan University of Technology, China

Managing the rapidly growing scholarly corpus poses significant challenges in representation, reasoning, and efficient analysis. An ideal system should unify structured knowledge management, agentic planning, and interpretable execution to support diverse scholarly queries – from retrieval to knowledge discovery and generation – at scale. Unfortunately, existing RAG and document analytics systems fail to achieve all query types simultaneously. To this end, we propose AgenticScholar, an agentic scholarly data management system that integrates a structure-aware knowledge representation layer, an LLM-centric hybrid query planning layer, and a unified execution layer with composable operators. AgenticScholar autonomously translates natural language queries into executable DAG plans, enabling end-to-end reasoning over multi-modal scholarly data. Extensive experiments demonstrate that AgenticScholar significantly outperforms existing systems in effectiveness, efficiency, and interpretability, offering a practical foundation for future research on agentic scholarly data management.

Additional Key Words and Phrases: Agentic Data Management, LLM-driven Query Planning

Authors' Contact Information: Hai Lan, h.lan@uq.edu.au, School of Electrical Engineering and Computer Science, The University of Queensland, Brisbane, Queensland, Australia; Tingting Wang, tingting.wang@uq.edu.au, School of Electrical Engineering and Computer Science, The University of Queensland, Brisbane, Queensland, Australia; Zhifeng Bao, zhifeng.bao@uq.edu.au, School of Electrical Engineering and Computer Science, The University of Queensland, Brisbane, Queensland, Australia; Guoliang Li, liguoliang@tsinghua.edu.cn, Department of Computer Science and Technology, Tsinghua University, Beijing, China; Daomin Ji, d.ji@uq.edu.au, School of Electrical Engineering and Computer Science, The University of Queensland, Brisbane, Queensland, Australia; Ge Lee, ge.lee@uq.edu.au, School of Electrical Engineering and Computer Science, The University of Queensland, Brisbane, Queensland, Australia; Feng Luo, f.luo@uq.edu.au, School of Electrical Engineering and Computer Science, The University of Queensland, Brisbane, Queensland, Australia; Zi Huang, huang@itee.uq.edu.au, School of Electrical Engineering and Computer Science, The University of Queensland, Brisbane, Queensland, Australia; Hailang Qiu, helloqiu@whu.edu.cn, School of Computer Science, Wuhan University, Wuhan, Hubei, China; Gang Hua, huagang01@whut.edu.cn, School of Computer Science and Artificial Intelligence, Wuhan University of Technology, Wuhan, Hubei, China.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2836-6573/2026/6-ART131

<https://doi.org/10.1145/3802008>

ACM Reference Format:

Hai Lan, Tingting Wang, Zhifeng Bao, Guoliang Li, Daomin Ji, Ge Lee, Feng Luo, Zi Huang, Hailang Qiu, and Gang Hua. 2026. AgenticScholar: Agentic Data Management with Pipeline Orchestration for Scholarly Corpora. *Proc. ACM Manag. Data* 4, 2 (SIGMOD), Article 131 (June 2026), 54 pages. <https://doi.org/10.1145/3802008>

1 Introduction

The exponential growth of scholarly corpora represents one of today’s most complex data management challenges [37]. This vast corpus is far more than a collection of text files; it spans **multiple modalities**, including but not limited to tables, figures, code snippets, and bibliographic metadata, forming an intricately interconnected and semi-structured ecosystem [48]. Managing and reasoning over this ecosystem is key to democratizing knowledge and promoting evidence-based research, yet it pushes the boundaries of traditional data management systems.

An Example Illustrating Scholarly Query Characteristics

In Figure 1, a researcher begins with an open question – “How has research on vector search been evolving, and what are promising yet unexplored problems since 2021?” – and the system must progressively clarify intents and compose a series of analytical steps to generate an evidence-grounded answer. We observe that typical scholarly queries often:

1. *Start open-ended and evolve diverse intents.* The researcher may begin with a general trend analysis on “vector search” and then diverge into different paths: stopping at the trend (**Intent 1**); expanding to performance comparison and research idea exploration across the full literature (**Intent 2**); or focusing on milestone papers for a targeted synthesis (**Intent 3**).
2. *Require multi-step semantic reasoning.* Addressing Q4 involves multiple analytical steps: first extract experimental results and limitations from individual papers, then rank the results and summarize shared limitations across papers, and finally generate a coherent response.
3. *Depend on multi-modal evidence and structured synthesis.* Resolving Q4 entails extracting quantitative results from tables (e.g., Table 5 of Starling) and figures (e.g., Figures 6 and 8 of SPANN, Figure 5 of Starling), aligning them across papers into comparative matrices, and grounding the synthesis in this integrated, multi-modal evidence.
4. *Require context-aware knowledge generation.* Addressing Q5 requires generating novel insights that extend beyond summarization. The system must interpret the problem context, analyze the methodological landscape, and infer which problem–method combinations remain underexplored to produce an informed and contextually grounded conclusion.

These query characteristics present a fundamental challenge for existing systems [46, 62, 73, 77], which will be discussed shortly in Section 2.2. While they may address isolated queries in Figure 1, they cannot autonomously orchestrate a complete analytical pipeline – from trend analysis (Q1) through multiple reasoning steps – to the ultimate stage of research idea exploration (Q5). This gap between capability and necessity motivates our introduction of AgenticScholar, **an agentic scholarly data management system** that autonomously interprets user intents, plans complex analytical pipelines, and executes them end-to-end within the rich semantic context of a scholarly ecosystem. We distill four key challenges for an agentic scholarly data management system:

Challenge 1: Semantic Representation and Contextualization of Scholarly Data. Scholarly documents contain multiple modalities that collectively convey a paper’s semantics. The challenge is not recognizing this diversity, but representing and indexing it in a way that preserves *both internal structure and semantic connections across components within a paper and those across related papers*. Queries like Q5 in Figure 1 demand reasoning at the topic level, integrating contextual evidence to perform comparative or synthetic analysis. Supporting them requires representations that maintain

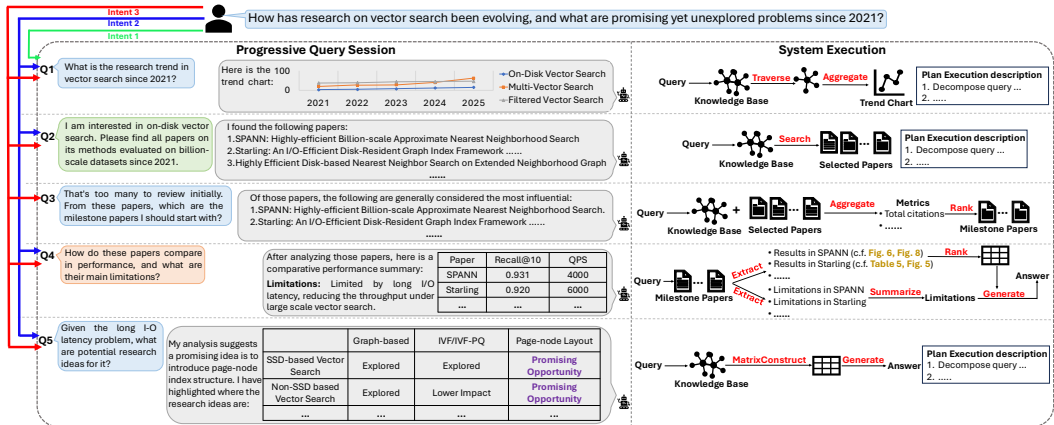


Fig. 1. The workflow of an agentic scholarly data management system, illustrating how it responds to diverse query intents.

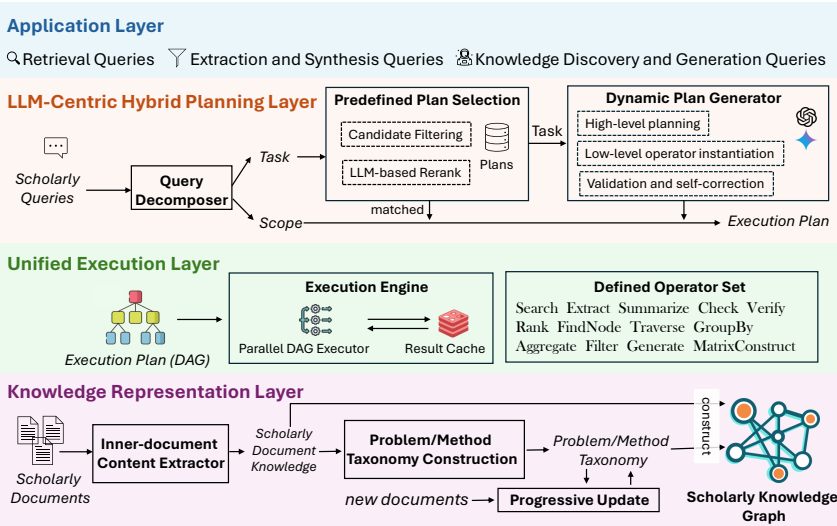


Fig. 2. System architecture of AgenticScholar.

contextual organization beyond individual papers. Efficiently constructing and maintaining such structures, while enabling fine-grained retrieval and reasoning, remains an open data management problem. Existing systems [77, 82] typically flatten documents into unstructured text or dense embeddings, discarding structure and context essential for complex scholarly analysis.

Challenge 2: Planning Complex Analytical Pipeline. As shown in Figure 1, scholarly queries exhibit *diverse intents* that go far beyond the rigid patterns of SQL or keyword search. A central challenge is to translate high-level natural language into an accurate and efficient execution plan. The system must infer user intent, decompose it into a *coherent sequence of analytical steps*, and determine an efficient execution strategy amid the uncertainty of operating over multi-modal data. Traditional query optimizers – built for fixed relational schemas – lack the flexibility needed for such dynamic and context-aware semantic reasoning.

Challenge 3: Generality and Extensibility of the Execution Framework. The space of scholarly queries is vast and continually evolving. As shown in Figure 1, a researcher’s exploration often begins with an open question and progresses through diverse queries. A system built on hard-coded, query-specific pipelines would be brittle and incapable of adapting to new analytical

needs. The challenge, therefore, is to design a framework that *balances generality with extensibility*: (1) its core set of analytical steps must be sufficiently expressive and composable to support a broad spectrum of queries without requiring custom implementations for each new one; and (2) its architecture must allow new analytical steps and execution plans to be integrated seamlessly, enabling support for emerging query types without necessitating a full system redesign.

Challenge 4: Interpretability and Traceability. For research-oriented data management, correctness alone is not enough – researchers need to understand and trust how an answer was produced. The challenge lies in exposing interpretable reasoning chains that make internal decisions explicit, including applied analytical steps, intermediate results, and evidence propagation. This requires reasoning processes to be materialized and inspectable, akin to query execution plans in DBMSs. As illustrated in Figure 1, queries may include reasoning traces – for instance, Q1 shows its execution plan, while Q4 cites supporting tables and figures.

Our Contributions – A Holistic Architecture for Agentic Scholarly Data Management. To address the above challenges collectively, we introduce AgenticScholar, which embodies a holistic system architecture (Figure 2) to unify knowledge representation, LLM-centric hybrid planning, and unified execution to support diverse scholarly queries.

- **Knowledge Representation Layer** (Section 3). To address *Challenge 1*, we introduce a *taxonomy-anchored knowledge graph* that unifies fine-grained document structure with an explicit conceptual organization of the research landscape. Unlike existing academic knowledge graphs [64, 81, 83] that primarily organize papers via bibliographic relationships, our representation models papers together with their internal components (e.g., sections, tables, figures, and experimental metadata), enabling fine-grained evidence access during query execution. Crucially, the graph is anchored by *hierarchical taxonomies* of research problems and methods, which provide shared semantic abstractions and serve as the contextual backbone for topic-level analysis, such as research trend analysis and research idea exploration. To support this capability, we develop a novel algorithm to *automatically construct* these taxonomies from the corpus and *progressively update and refine* them as new research outputs emerge.

- **LLM-Centric Hybrid Query Planning Layer** (Section 4). To tackle *Challenge 2*, this layer places an *LLM at the center of query planning* to interpret diverse user intents. Unlike traditional query optimizers in DBMSs, the planner integrates *LLM-driven semantic reasoning* to interpret ambiguous natural language queries, decompose analytical intents, and reason about operator composition beyond relational algebra. To reduce planning cost while ensuring reliable performance, the planner adopts a *hybrid planning framework* that combines reusable predefined plans for common scholarly queries with dynamic plan generation for queries outside this library. *Explicit plan validation and self-correction* are further incorporated to ensure plan executability.

- **Unified Execution Layer** (Section 5). To address *Challenge 3*, we introduce the Unified Execution Layer, which serves as the operational backbone of AgenticScholar. First, we analyze typical scholarly queries in Figure 3 and design a set of *general, reusable operators* that can be composed to support a wide range of analytical goals (see Section 5.1). Second, we design a *parallel execution engine* with *result caching* to accelerate query processing by reusing (intermediate) results across queries (see Section 5.2). The architecture remains extensible, allowing new operators to be seamlessly integrated as emerging query types arise. To ensure *interpretability and traceability* (Challenge 4), the execution engine materializes the full query plan and data lineage for each result, and exposes them to users for inspection. Moreover, users can control the level of detail in the generated outputs through internal operator parameters – for example, requesting a concise summary, a detailed explanation, or a fully evidence-backed reasoning trace.

Tier 3: Knowledge Discovery and Generation Queries

✓ **Research trend analysis:** *What is the research trend on vector search since 2020?*

✓ **Research idea exploration:** *What are the under-explored combinations of problems and methods in the field of learned index on spatial index?*

✓ **Milestone paper selection:** *Please list the five most important papers in the learned index in terms of the novelty in the proposed method and their impact.*

Tier 2: Information Extraction and Synthesis Queries

on Single Document

on Multiple Documents

✓ **Extract:**

Experiment setting and result, problem definition, method

/

✓ **Summarize:**

Impact of parameters

Experiment setting, pros and cons of methods, differences of problem definitions

✓ **Rank:**

Rank the variants of the proposed methods

Experiment results on common datasets and workloads

✓ **Check:**

Inconsistencies of used datasets and metrics to their origin ones

Inconsistencies experiment results on common datasets and workloads

Tier 1: Paper Retrieval Queries

✓ **Search the papers (scope)** that satisfy a set of constraints on paper metadata, problems, et al.
Find papers on vector search since 2023 that use graph-based methods.

Fig. 3. Representative scholarly queries across three tiers.

• **System Implementation and Evaluation** (Section 6). Together, these layers form a unified framework that addresses the four challenges outlined above and supports diverse, open-ended scholarly queries. We implement AgenticScholar and conduct comprehensive experiments to evaluate its capabilities across retrieval, information extraction and synthesis, and knowledge discovery and generation queries. Furthermore, we present an end-to-end case study simulating realistic, open-ended scholarly exploration and perform ablation studies to quantify the impact of key architectural choices.

2 Typical Queries and Related Work

2.1 Typical Scholarly Queries

Figure 3 shows representative scholarly queries in AgenticScholar.

Tier 1: Paper Retrieval Queries. This foundational tier focuses on locating papers matching specific criteria, serving as the basis for subsequent analytical tasks.

Tier 2: Information Extraction and Synthesis Queries. This tier involves fine-grained content analysis, covering both *single-document extraction* to extract details of a paper and *multi-document synthesis* to synthesize knowledge across papers.

Tier 3: Knowledge Discovery and Generation Queries. This advanced tier extends beyond retrieval or summarization to generate novel insights and actionable guidance.

2.2 Related Systems

Several systems support scholarly queries to a certain extent, as shown in Table 1. We categorize these systems into four classes according to their design focus and depth of reasoning.

Scholarly-Centric Systems. This class of systems [11, 23] supports large-scale paper retrieval and literature summarization. Among them, Elicit [23] covers all tiers of scholarly queries but offers only limited query refinement – restricted to improving existing summaries – and often incurs longer response times. Other systems are designed for a single query tier. For example, ChatPDF [13] allows users to upload papers for information extraction and synthesis, whereas PASA [31] targets paper retrieval by leveraging textual and citation-based signals to locate studies.

Table 1. Comparisons on related systems (● supported; ◐ partially supported; × not supported).

System	Tier 1 Queries	Tier 2 Queries		Tier 3 Queries		
	Retrieval	Single doc.	Multiple doc.	Trend Analysis	Idea Exploration	Milestone Selection
Elicit [23]	●	●	●	●	●	●
Ai2 Asta ¹ [11]	●	×	×	×	×	×
ChatPDF [13]	×	●	×	×	×	×
RAG [26]	×	●	●	×	×	×
Gemini Deep Research [66]	◐	●	●	●	●	●
Tongyi Deep Research [67]	◐	●	●	●	●	●
AgenticData [77]	×	●	●	×	×	×
AgenticScholar	●	●	●	●	●	●

¹Ai2 Asta claims to support literature summarization but currently fails on complex Tier-2 and Tier-3 queries based on our test; hence marked as not supported.

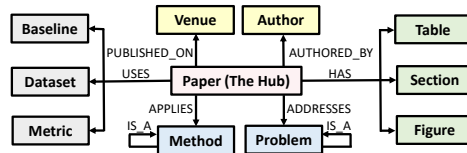


Fig. 4. The schema of the knowledge graph.

LLM-based Tools. Retrieval-Augmented Generation (RAG) [22, 26, 30, 35, 41, 50, 85] enhances LLMs by grounding their outputs in retrieved sources. This approach improves reliability in single-document extraction and multi-document synthesis, providing a scalable alternative to direct LLM APIs [5–8, 27], which are constrained by token limits and high computational costs. However, most existing RAG implementations retrieve only a limited number of content chunks, resulting in poor source coverage and weak contextual integration. Consequently, they are less effective for open-ended queries that require progressive exploration and deep reasoning across corpora.

Agentic Deep Research Systems. Representative applications, such as Gemini Deep Research [66] and Tongyi Deep Research [67], primarily target Tier-3 queries. Unlike standard LLM APIs, they autonomously plan multi-step reasoning – decomposing complex questions, extracting real-time information, and synthesizing structured, citation-grounded reports. This enables higher-level capabilities like identifying research trends, milestone papers, and promising ideas. However, each presents trade-offs: Gemini Deep Research produces more comprehensive results but with longer response times, reducing interactivity, whereas Tongyi Deep Research may respond faster but is less effective for nuanced higher-tier queries.

Semi-Structured Document Analytics Systems. Recent systems [12, 46, 47, 62, 69, 73, 77, 79, 82] extend LLM-based reasoning to document analytics but mainly support Tier-2 queries. They struggle with Tier-3 queries, which are open-ended and involve evolving user intents, due to their reliance on rigid queryable forms such as manually written programs [47, 62, 73] or SQL-like queries [46, 79]. While systems such as AgenticData [77] and Unify [82] introduce natural language interfaces, their data models remain insufficient for multi-modal and context-aware representation, and their limited operator spaces restrict their ability to perform complex semantic reasoning, such as research idea exploration.

3 Knowledge Representation Layer: From Multi-Modal Structure to Hierarchical Research Taxonomies

Supporting the diverse queries shown in Figure 3, especially Tier-3 queries that require reasoning over research topics, demands a storage representation that goes beyond document-centric storage.

Such a representation must both expose the fine-grained, multi-modal structure of individual papers and anchor them within a shared, explicit conceptual organization of the research landscape.

Thus, the representation of a scholarly corpus must satisfy three key requirements: (1) provide fine-grained access to multi-modal content such as sections, tables, and figures; (2) maintain explicit links among related elements – including datasets, metrics, baselines, and citations – to capture experimental and bibliographic relationships; and (3) organize papers in structured hierarchies of problems and methods, enabling reasoning over research topics rather than isolated documents.

Taxonomy-Anchored Knowledge Graph. To this end, AgenticScholar introduces a *Knowledge Representation Layer* that transforms the scholarly corpus into a structured knowledge graph (KG). While prior academic KGs such as MAG [83], OpenAlex [64], and AMiner [81] primarily model bibliographic relationships among papers, authors, venues, and citations, such document-centric representations are insufficient for supporting topic-level and compositional reasoning. As a result, our KG is anchored by *Taxonomy Nodes* – specifically Problem and Method nodes. These nodes define shared abstractions of research problems and algorithmic strategies, serving as first-class semantic primitives for reasoning. For instance, when addressing Q5 in Figure 1, the system can directly infer unexplored problem–method combinations by traversing the problem and method taxonomies within the vector search domain, rather than aggregating evidence across individual papers, thereby enabling more efficient and scalable reasoning.

As illustrated in Figure 4, our KG is structured *around these taxonomy nodes* to capture the multi-modal structure of scholarly documents. Each Paper node acts as a central hub that links to Content Nodes (including Section, Figure, and Table), Experimental Context Nodes (including Dataset, Metric, and Baseline), and Bibliographic Nodes (including Author and Venue). This joint modeling of taxonomy-level abstractions and document-level evidence enables AgenticScholar to support complex Tier-3 queries that require topic-level reasoning grounded in document content.

As the KG instantiation follows engineering practices, we provide implementation details in Appendix B.1; the following section presents our novel algorithm for taxonomy construction.

3.1 Reference-enhanced Taxonomy Construction

Constructing high-quality taxonomies for a scholarly corpus remains a significant challenge. While various approaches exist, they face several limitations: (1) Corpus-driven methods [15, 32, 36, 72, 74, 91, 96] build hierarchies directly from papers. While effective at capturing domain-specific nuances, they often suffer from data sparsity, yielding fragmented or overly narrow structures [49]. (2) LLM-driven methods [14, 36, 78, 90] leverage an LLM’s internal knowledge to ensure broader conceptual coverage. However, they frequently fail to capture emerging topics that appear in the corpus but are absent from the model’s pre-training data. (3) Crucially, both paradigms typically lack mechanisms for progressive refinement and incorporating newly papers may require reconstructing the entire taxonomy from scratch – a prohibitive cost for rapidly evolving research fields.

Our Novel Solution. To overcome these limitations, we propose a *Reference-enhanced Taxonomy Construction Framework*. The framework synergizes an LLM-derived scaffold with corpus-grounded evidence, ensuring that the resulting taxonomy is structurally comprehensive and faithful to the target corpus. Crucially, it supports *progressive refinement*, allowing the taxonomy to evolve dynamically as new papers arrive, without full reconstruction. As outlined in Algorithm 1, the taxonomy construction proceeds in four stages:

- Structured Information Extraction: Extract key aspects that characterize paper’s problem or method.
- Cross-Paper Standardization: Synthesize extracted aspects across papers into standardized concepts by identifying recurring concepts and unifying synonyms.

Algorithm 1 TAXONOMYCONSTRUCTION

Input: scholarly corpus D , optional new papers D_{new} , refinement factor α , match threshold τ_{match} ;

Output: taxonomy T ;

```

// Stage 1: Structured Information Extraction
1:  $\mathcal{A} \leftarrow \{\}$ ; ▷ map: paper  $\mapsto$  problem/method aspects
2: for  $p \in D$  do  $\mathcal{A}[p] \leftarrow \text{EXTRACTASPECT}(p)$ ;
// Stage 2: Cross-paper Standardization
3:  $C \leftarrow \text{CROSSPAPERSTANDARDIZATION}(\mathcal{A})$ ;
// Stage 3: Reference Taxonomy Generation and Alignment
4:  $L \leftarrow \text{INFERTOPICLABEL}(D)$ ,  $T_{\text{ref}} \leftarrow \text{GENERATEREFERENCETAXONOMY}(L)$ ;
5:  $T \leftarrow \text{COPYTREE}(T_{\text{ref}})$ ;
6: for  $(C, D_C) \in C$  do
7:    $T.u \leftarrow \text{TOPDOWNLOCATEPARENT}(T.\text{root}, C)$ ;
8:    $T.v \leftarrow \text{FINDMATCHINGCHILD}(T.u, C, \tau_{\text{match}})$ ;
9:   if  $T.v == \text{NULL}$  then  $T.v \leftarrow \text{ADDCHILD}(T.u, C)$ ;
10:  Assign papers  $D_C$  to  $T.v$ ;
// Stage 4: Progressive Update and Branch Refinement
11: if  $D_{\text{new}} \neq \emptyset$  then
12:    $\mathcal{A}_{\text{new}} \leftarrow \{\}$ ;
13:   for  $p \in D_{\text{new}}$  do
14:      $A \leftarrow \text{EXTRACTASPECT}(p)$ ,  $C \leftarrow \text{MAPTOCONCEPTSORCREATE}(A)$ ;
15:      $T.u \leftarrow \text{TOPDOWNLOCATEPARENT}(T.\text{root}, C)$ ;
16:      $T.v \leftarrow \text{FINDMATCHINGCHILD}(T.u, C, \tau_{\text{match}})$ ;
17:     if  $T.v == \text{NULL}$  then  $T.v \leftarrow \text{ADDCHILD}(T.u, C)$ ;
18:     Assign paper  $p$  to  $T.v$ ;
19:     if  $\text{NEWPAPERCOUNT}(T.v) \geq \alpha$  then  $\text{REFINEBRANCH}(T.v)$ 
20: return  $T$ 

```

• Reference Taxonomy Alignment: Align the extracted aspects of each paper and the standardized concepts with an LLM-generated reference taxonomy, constructed using topic labels from the corpus, to produce a taxonomy that integrates corpus-grounded evidence with domain knowledge.

• Progressive Update and Branch Refinement: Incorporating newly arriving papers by progressively refining the taxonomy structure through three operations: assigning them to existing nodes, creating new nodes, or introducing new branches.

In the following, we describe each stage, referencing Algorithm 1 and the illustrative example in Figure 5. We use the construction of the problem taxonomy as a representative case, as the construction processes for problem and method taxonomies are analogous. Implementation details are provided in Appendix B.2.

Stage 1: Structured Information Extraction. The first stage extracts essential aspects from each paper in the scholarly corpus to facilitate alignment (Line 2 in Algorithm 1). We adopt a context-aware extraction strategy that combines *Shared Content* (e.g., Abstract, Introduction, and Related Work), which orients the LLM to the paper’s overall scope, with *Targeted Content* (e.g., the Problem Definition for the problem taxonomy), enabling precise extraction of problem- or method-specific information. The extracted aspects differ by taxonomy type:

- For problem taxonomy, the aspects consist of formal specifications of the *input* and *output*. As illustrated in Figure 5, this is achieved by decomposing each problem into concrete data constraints (e.g., “Input: a query vector and an attribute filter” in Paper 1) and objectives (e.g., “Output: top- k candidates” in Paper 1). Such formalization enables fine-grained distinctions between closely related problems, e.g., differentiating KNN retrieval (Paper 2) from range search (Paper 3).

- For method taxonomy, the aspects include *key techniques* (e.g., core components, ordered workflows), and, crucially, the *strengths and weaknesses*. While technical descriptions capture how a method operates, strengths and weaknesses explain why it was designed, enabling methods to be

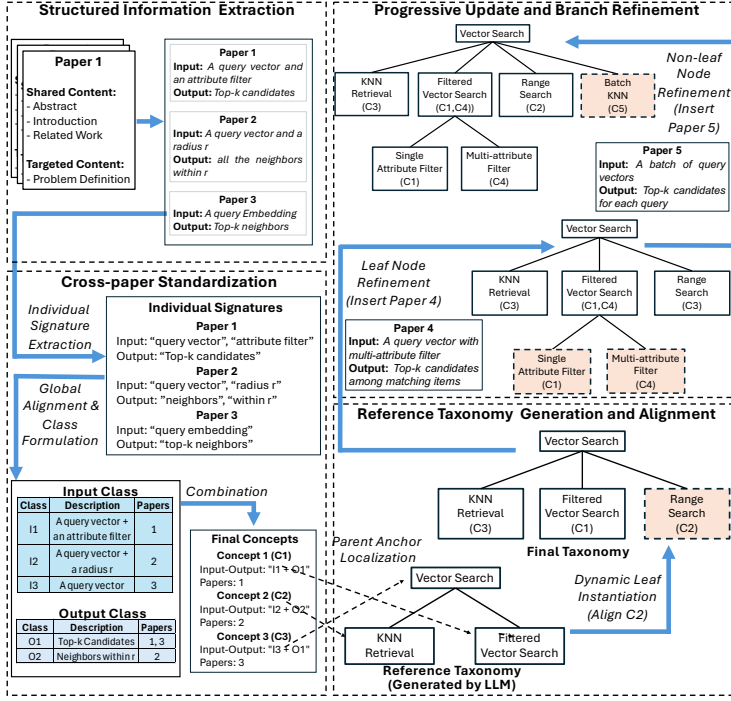


Fig. 5. Reference-enhanced taxonomy construction.

grouped not only by mechanics but also by shared optimization goals (e.g., trading accuracy for speed). This reveals implicit methodological families that are often missed by technical descriptions.

Stage 2: Cross-Paper Standardization. While the previous stage yields isolated aspects, this stage synthesizes them into coherent, corpus-level concepts (Line 3 in Algorithm 1). We employ a three-step process to standardize extracted aspects – unifying diverse phrasings into canonical classes, synthesizing these classes into composite concepts, and assigning each paper to the concept, which serves as corpus-ground evidence for the next stage.

Step 1: Individual Signature Extraction. We extract key noun phrases using an LLM, which serve as semantic signatures for the target aspect. As shown in Figure 5, when the input is the target aspect, Paper 1 yields signatures “query vector” and “attribute filter”. These signatures capture essential semantics required for alignment.

Step 2: Global Alignment and Class Formation. We prompt an LLM to identify semantically equivalent signatures and group them under a canonical class. As shown in Figure 5, the distinct signatures “top-k candidates” (Paper 1) and “top-k neighbors” (Paper 3) are identified as synonyms and consolidated into Output Class O_1 .

Step 3: Assignment and Concept Formulation. Canonical classes derived from aspects are combined to form *concepts*, which represent abstractions of problem families (e.g., pairing Input Class I_1 with Output Class O_1), and each paper is mapped to the most appropriate concept. As shown in Figure 5, Paper 1 is assigned to Concept C_1 (representing $I_1 + O_1$), while Paper 2 is assigned to Concept C_2 . Papers that do not match existing concepts form new subgroups that are recursively re-processed through Step 1 and Step 2, ensuring comprehensive coverage.

Stage 3: Reference Taxonomy Generation and Alignment. After standardizing corpus-grounded concepts, this stage integrates an LLM-derived reference taxonomy with these concepts to construct the final taxonomy, which forms the core of our framework. This integration are in three steps.

Step 1. Reference Taxonomy Generation. We first instruct an LLM to infer a topic label from the corpus and generate a skeletal reference taxonomy (Line 4 in Algorithm 1). In Figure 5, the LLM identifies topic “Vector Search” and generates child nodes “KNN Retrieval” and “Filtered Vector Search” based on general domain knowledge.

Step 2. Node Localization. We then map each standardized concept to the reference taxonomy through two sub-steps.

(1) *Parent Anchor Localization:* We traverse the reference taxonomy in a top-down manner to identify the most specific parent node that subsumes the concept (Line 7 in Algorithm 1). In Figure 5, for Concept C_2 , the system identifies the root node “Vector Search” as the parent anchor, since C_2 does not semantically fit under the existing “KNN Retrieval” or “Filtered Vector Search” nodes.

(2) *Dynamic Leaf Instantiation:* If the concept does not match any existing child of the parent anchor, we instantiate a new node (Lines 8-9 in Algorithm 1). In Figure 5, because a specific “Range Search” node is absent from the reference taxonomy, the system dynamically creates a new node for C_2 , prompts the LLM to generate a canonical name “Range Search”, and attaches it to the taxonomy.

Step 3. Paper Assignment. Finally, all papers associated with the concept are linked to the corresponding taxonomy node (Line 10 in Algorithm 1). For example, Paper 2 is assigned to “Range Search” node, while Paper 1 is assigned to “Filtered Vector Search”, ensuring that every document is reachable through the taxonomy hierarchy.

Stage 4: Progressive Update and Branch Refinement. To accommodate the dynamic nature of research, our framework supports progressive update, allowing the taxonomy to evolve as papers arrive *without requiring reconstruction from scratch* (Lines 11-20 in Algorithm 1). Specifically, new papers are first processed by extracting aspects following Stage 1 and mapping them to existing concepts or creating new concepts (Line 14 in Algorithm 1). They are then assigned to the most specific existing taxonomy node via Step 2 and Step 3 of Stage 3 (Lines 15-18 in Algorithm 1). To prevent taxonomy nodes from becoming overly broad due to concept drift, we further employ a branch refinement mechanism.

Step 1. Refinement Trigger. As shown in Line 19 of Algorithm 1, when the number of new papers linked to a node exceeds a predefined threshold α , the node is flagged for refinement.

Step 2. Refinement Execution. The refinement strategy adapts based on the node’s position in the taxonomy and leverages corpus-grounded clustering to discover fine-grained structures that are often absent from the LLM’s pre-training data.

Case 1: Leaf Node Refinement. This case captures the emergence of granular subtopics. As illustrated in Figure 5, the leaf node “Filtered Vector Search” accumulates new papers, including Paper 4, which introduces multi-attribute filtering. The system performs semantic clustering on these papers and splits the node into a more detailed subtree consisting of “Single Attribute Filter” (C_1) and “Multi-attribute Filter” (C_4).

Case 2: Non-leaf Node Refinement. This case handles papers that fit a high-level category but do not align with any child nodes. In Figure 5, Paper 5 maps to the root “Vector Search” but fits none of its children. The refinement process identifies this distinct cluster and instantiates a new sibling branch, “Batch KNN” (C_5), extending the taxonomy breadth without disrupting existing branches.

4 LLM-Centric Hybrid Planning Layer: From Natural-Language Queries to Executable Plans

The Query Planning layer translates high-level natural-language queries into executable Directed Acyclic Graphs (DAGs) of operators. While this objective shares the high-level spirit of classical database query optimization [40, 71], our system operates in a fundamentally different setting.

Unique Challenges. Traditional query optimizers operate over algebraic SQL queries with well-defined schemas and formally specified operator semantics. In contrast, our planner must reason over open-ended natural language scholarly queries, which introduces challenges absent in DBMS planning: (1) *Lack of algebraic semantics*: Natural language queries do not admit a fixed algebraic representation, rendering rule-based transformations and cost-based enumeration ineffective. (2) *Open-ended analytical intents*: Scholarly queries are often open-ended and may evolve into diverse analytical intents (Figure 1), leading to a vast plan space that extends well beyond relational algebra. (3) *Verification complexity*: Unlike deterministic SQL compilation, generative planning requires explicit mechanisms to validate plan correctness and executability before execution.

Our Planner. To address these challenges, we propose an *LLM-centric hybrid query planner* that places LLMs at the core of semantic reasoning while carefully constraining their role to ensure efficiency and reliability. Our planner is built on three key principles:

- *LLM-driven Semantic Reasoning.* We leverage LLMs to interpret ambiguous natural language queries, decompose analytical intents, and reason about operator composition.
- *Hybrid Planning for Efficiency and Generality.* To reduce planning cost while ensuring reliable performance, we combine a library of predefined plans for common scholarly queries with dynamic plan generation for queries outside this library.
- *Robustness through Validation and Self-correction.* We introduce explicit plan validation and an iterative self-correction loop to mitigate the non-determinism and errors of LLM-generated plans.

Figure 6 illustrates the end-to-end process by which a natural language query is translated into a validated execution plan; the details are presented in the following subsections.

4.1 LLM-Driven Query Decomposition

To bridge the gap between ambiguous natural language and modular execution, our system first decomposes the analytical intent into two components: a Scope and a Task. The Scope specifies the entities or conceptual boundary of analysis, while the Task describes the analytical step to be performed over that scope.

This decomposition is achieved by prompting an LLM with a structured instruction that explicitly separates “what to analyze” from “how to analyze it”. As shown in Figure 6, given the query “Find papers on vector search since 2023 that use graph-based methods and build a table comparing their indexing speed and memory usage”, the LLM produces: (1) Scope – “papers on vector search since 2023 that use graph-based methods”; and (2) Task – “build a table comparing their indexing speed and memory usage”.

This decomposition yields a clearer and more modular representation of analytical intents, reducing ambiguity and enabling subsequent planning stages to operate more reliably and efficiently.

4.2 Predefined Plan Selection

For common and well-understood scholarly queries (Figure 3), invoking a fully generative planner is inefficient. To achieve low latency and high reliability for such queries, our system first attempts to match the query’s Task component to a plan in a *Predefined Plan Library* (Appendix C). This library plays a role analogous to cached query plans in traditional DBMSs [2, 21, 53]; however, despite this resemblance, our design differs in two fundamental ways to accommodate the semantic nature of natural-language queries:

- **Natural-language-driven Parameters.** In our setting, plan “parameters” are not SQL-like constants (e.g., WHERE id = ?) but semantic intents expressed in natural language (e.g., “Extract indexing speed and memory usage”). Consequently, plan selection requires semantic understanding rather than syntactic matching.

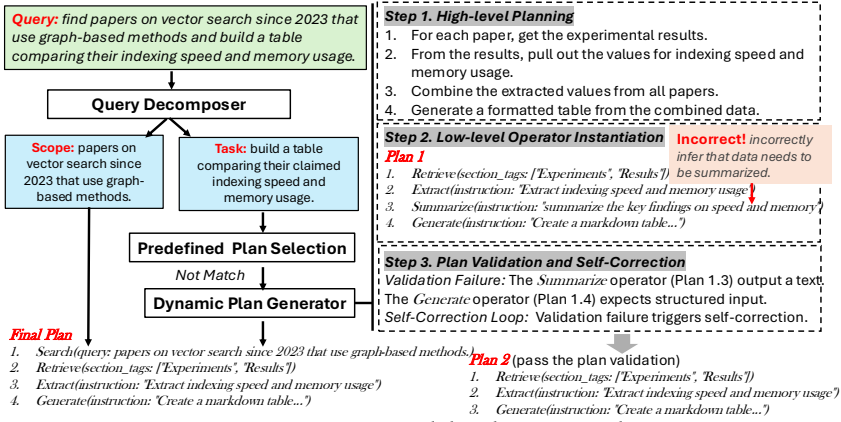


Fig. 6. From query to validated execution plan.

• **Reliability-oriented plan selection.** As the plan library is extensible and may grow large over time, directly prompting an LLM to select from all available plans is unreliable: long prompts with many candidates often lead to misselection. To ensure robust plan reuse, we adopt a two-stage selection strategy:

(1) *Candidate Filtering*: A semantic search is first performed to compare the task description against the descriptions of all plans in the library, retrieving a small candidate set of plans.

(2) *LLM-based Reranking and Validation*: The candidate plans are then evaluated by a reasoning LLM, which assesses their alignment with the analytical intent and assigns a confidence score.

A plan is selected only if its confidence score exceeds a high threshold (e.g., > 90%); otherwise, the query is treated as ad-hoc and passed to the dynamic plan generator. For example, the task “Extract indexing speed and memory usage” directly matches a predefined extraction plan specialized for experiment sections.

This design enables our system to retain the low latency benefits of plan reuse while preserving the semantic flexibility and robustness required for natural language scholarly queries.

4.3 Dynamic Plan Generation with Self-correction

For queries that fall outside the predefined library, our system invokes the *Dynamic Plan Generator*. To manage the combinatorial explosion of the plan space and reduce hallucinations, we employ a decomposed generation process: generating a high-level logical plan via few-shot demonstrations, followed by low-level operator instantiation. Crucially, we introduce a **self-correction loop** to ensure robustness.

Step 1. High-Level Planning with Demonstrations. To narrow the search space, our system first generates a high-level logical plan as an abstract sequence of steps. To guide the LLM, we retrieve representative (query, plan) demonstrations from an extensible library via semantic search and inject them into the LLM context.

Step 2. Low-Level Operator Instantiation. Our system then proceeds to the low-level instantiation stage. It iterates through each abstract step of the logical plan and makes a series of focused, less complex calls to the LLM. For each step, the LLM’s task is to select the single most appropriate operator from the operator library and fully parameterize it to accomplish that specific sub-task. This enables our system to ask the LLM to solve a much simpler problem at each step rather than a complex problem of generating an entire DAG at once, significantly increasing the success rate.

Step 3. Robustness through Plan Validation and Self-Correction. Generated plans are never trusted implicitly. Before execution, every plan is passed to a *Plan Validator* that performs fast,

Table 2. Summary of Operator Library.

Category	Operator	Description
Knowledge Access	Search	Locate documents from the knowledge graph
Knowledge Navigation	FindNode	Locate specific entry-point nodes within taxonomies
	Traverse	Traverse the knowledge graph from a set of starting nodes
	Retrieve	Fetch the raw content of specific sections from a document
Knowledge Generation	MatrixConstruct	Create a structured matrix summarizing problems and methods
	Generate	Generate new content based on inputs and specific instructions
Semantic Content Processing	Extract	Extract specific, structured information from raw text content
	Summarize	Summarise textual information into a concise, coherent overview
	Check	Compare information across multiple peer documents
	Verify	Verify a specific claim against a designated source of evidence
	Rank	Order a set of entities based on a specific, measurable criterion
Relational-Style Data Manipulation	GroupBy	Partition a list of entities based on a specified attribute
	Aggregate	Compute summary statistics over a set of entities
	Filter	Return a subset of entities that satisfy the condition

deterministic checks for DAG validity, operator correctness. If a plan fails validation, or if a syntactically valid plan fails at runtime, our system triggers a *self-correction loop*. It is re-invoked with an augmented context that includes the failed plan and the specific error message. The prompt is shifted from “planning” to “debugging”, instructing the LLM to analyze the error and propose a corrected plan. As shown in Figure 6, the Summarize operator is ineffective because the subsequent Generate operator requires structured numeric evidence to construct the table. However, Summarize produces only high-level textual descriptions, which likely omit the detailed quantitative information needed for table generation.

Final Plan Composition. The final stage assembles the complete execution plan by composing the plan for the Scope component with the plan for the Task component. Our system chains these two parts together, ensuring the output of the initial operator (e.g., Search or FindNode from the scope analysis) is correctly fed as the input to the subsequent task plan (either the selected predefined plan or the dynamically generated one). The result is an DAG passed to the execution engine.

5 Unified Execution Layer: Operator Orchestration and Execution

As established in Section 2.1, scholarly queries span a wide spectrum from simple document retrieval to advanced knowledge discovery and generation. A system designed to support this diversity, along with future analytical needs, requires a *unified and extensible* execution framework rather than a collection of hard-coded, query-specific pipelines. To achieve this, we first analyze the typical query patterns and our underlying knowledge representation to design a library of composable operators (Section 5.1). Following established principles from traditional database systems [42, 54, 63], these operators serve as the building blocks for execution plans, which are structured as Directed Acyclic Graphs (DAGs). We build an execution engine that orchestrates these DAG-based plans to deliver accurate and interpretable results (Section 5.2).

5.1 Operator Library

The operator library in AgenticScholar provides building blocks for execution plans and is summarized in Table 2. We introduce novel operators tightly coupled with the taxonomy-anchored knowledge graph, enabling topic-level reasoning to support Tier-3 queries beyond retrieval and summarization. Representative operators are described below, with the full details in Appendix D.

Search. This operator serves as the main entry point for retrieving relevant scholarly documents from the knowledge graph. It supports natural language queries (e.g., Q2 in Figure 1) that combine precise constraints with abstract intent via a hybrid retrieval strategy balancing lexical precision

and semantic recall. Given a query, Search uses an LLM to decompose it into structured metadata constraints (e.g., title, authors, venue, year) and semantic intents over research aspects (e.g., topics, problems, methods, datasets). Metadata constraints are handled using BM25-based lexical retrieval [68], while semantic intents are matched via dense retrieval over BGE embeddings [88] indexed by FAISS [20]. Retrieved candidates are then refined by an LLM-based reranker that jointly reasons over the query, paper metadata, and KG-derived aspect summaries to produce aligned results for downstream operators.

FindNode. This operator identifies an entry-point taxonomy node for topic-level reasoning. It supports direct key-based lookup when an identifier is provided, or semantic matching over precomputed node description embeddings to select the best-matching concept.

Traverse. This operator enables controlled multi-hop navigation over the knowledge graph from given starting nodes. It uses a typed `traversal_path` to constrain relationship and entity types at each hop, preventing unintended expansion. At runtime, Traverse compiles the path into a multi-hop graph query (e.g., Cypher [24]) and returns an entity list for downstream operators such as Aggregate (e.g., Q1 in Figure 1) and Rank (e.g., Q3 in Figure 1).

MatrixConstruct. This operator supports research idea exploration (e.g., Q5 in Figure 1) by synthesizing a structured view of a research topic to reveal promising directions. Given a topic, MatrixConstruct uses problem and method taxonomies to identify representative problems and methods, and organizes them into a problem-method matrix. Each cell in the matrix aggregates supporting evidence (e.g., representative papers and summaries). The matrix can be inspected directly or passed to Generate for evidence-grounded synthesis, enabling identification of sparse or missing problem-method combinations.

Remark on Extensibility and Generality. The operator library is designed to be both general and extensible: *Generality* is achieved through two mechanisms: (1) a concise set of well-chosen operators and (2) flexible instruction parameters that allow each operator to adapt to a wide range of tasks without altering the system's core logic. *Extensibility* arises from the modular architecture, which enables new operators to be added seamlessly as analytical intents evolve. Together, these properties ensure that the system can scale and adapt in tandem with the changing research landscape.

5.2 Unified Query Execution Engine

The Unified Query Execution Engine realizes DAG-based query plans produced by the planner introduced in Section 4. It instantiates operators, resolves their dependencies, and manages intermediate and final results during execution. Its design emphasizes scalable parallel execution, efficient result reuse, and end-to-end traceability to support reliable and interpretable query execution. Algorithm 2 sketches the end-to-end execution workflow.

5.2.1 Execution Model. There are three key steps:

Step 1. Scope Execution and Plan Unfolding. The engine instantiates the query plan in two phases. Instead of materializing the entire DAG upfront, it first executes the operators in the query's Scope (e.g., Search; Line 1 in Algorithm 2) to obtain an initial result set (e.g., N papers). It then enters a plan unfolding stage by inspecting each operator's `execution_mode` (Line 2 in Algorithm 2). Operators marked as `instance` (e.g., Extract) are expanded into N parallel instances, one per item, while `group` operators (e.g., Summarize) are instantiated once to aggregate across all results. This process transforms the compact plan into a data-aware execution graph that explicitly exposes parallelism.

Algorithm 2 UNIFIEDQUERYEXECUTION

Input: query plan P ;

Output: final result R ;

Step 1: Scope Execution and Plan Unfolding

1: $S \leftarrow \text{EXECUTESCOPEOPERATORS}(P)$;

2: $G \leftarrow \text{UNFOLDPLAN}(P, S)$;

► Instantiate operators based on execution_mode

Step 2: Dependency Resolution

3: **for** operator $o \in G$ **do**

4: $\text{dep_count}[o] \leftarrow$ number of unresolved dependencies of o ;

5: **if** $\text{dep_count}[o] = 0$ **then** enqueue o into Q ;

Step 3: Operator Dispatch and Parallel Execution

6: **while** Q is not empty **do**

7: $o \leftarrow$ dequeue from Q

► May execute in parallel

8: $\text{inputs} \leftarrow \text{FETCHINPUTS}(o, \text{Buffer}, \text{Cache}, \text{Storage})$;

9: **if** $\text{CACHEHIT}(o, \text{inputs})$ **then** $\text{output} \leftarrow \text{LOADFROMCACHE}(o, \text{inputs})$;

10: **else** $\text{output} \leftarrow \text{EXECUTE}(o, \text{inputs}), \text{STOREINCACHE}(o, \text{inputs}, \text{output})$;

11: $\text{Buffer}[o] \leftarrow \text{output}$;

12: $\text{RECORDTRACE}(o, \text{inputs}, \text{output})$;

13: **for** downstream operator d of o **do**

14: $\text{dep_count}[d] \leftarrow \text{dep_count}[d] - 1$;

15: **if** $\text{dep_count}[d] = 0$ **then** enqueue d into Q ;

16: **return** outputs produced by terminal operators in G ;

Step 2. Dependency Resolution. Given the unfolded plan, the engine performs a topological analysis to determine execution order, enqueueing operators with no unresolved dependencies as ready for execution (Lines 3–5 in Algorithm 2).

Step 3. Operator Dispatch and Parallelism. Operators are dispatched once all required inputs are available. Upon completion, the engine propagates outputs, updates dependency counters of downstream operators, and enqueues operators whose dependencies are satisfied (Lines 6–15 in Algorithm 2). Multiple operators may become ready concurrently, enabling parallel execution while preserving dependency correctness.

5.2.2 Result Management, Reuse, and Traceability. During execution, AgenticScholar manages intermediate and final results through transient buffering, persistent caching, and execution traceability, enabling efficient reuse of computation while ensuring transparent and interpretable query execution.

Transient Buffering. Intermediate results are stored in a transient in-memory buffer keyed by operator IDs (Line 11 in Algorithm 2). Upon completion, each operator materializes its output into the buffer, from which downstream operators retrieve inputs by referencing upstream IDs. Operators without dependencies read directly from the storage layer (Line 8 in Algorithm 2), enabling efficient in-run reuse without redundant computation.

Persistent Caching. Beyond operator-level buffering, the engine maintains an execution cache to persist and reuse results across executions. Unlike transient buffers, the cache stores outputs of plans and subplans for reuse (Line 10 in Algorithm 2). When identical or overlapping plans are reissued, cached results are retrieved instead of recomputed (Lines 8–9 in Algorithm 2). Cached entries are persisted to disk, enabling durability across process lifetimes and accelerating iterative and incremental execution.

Execution Traceability. To support auditing and interpretation, all runtime artifacts are keyed by execution node IDs. The engine records per-node provenance and metadata, including dependencies, status transitions, and runtime statistics (Line 12 in Algorithm 2), enabling reconstruction and inspection of the annotated execution DAG that produced each result.

Table 3. Effectiveness on Tier-1 queries.

Methods	Factual Queries		Analytical Queries	
	R-Precision	MAP	R-Precision	MAP
BM25	0.49	0.52	0.42	0.45
GTR	0.44	0.47	0.36	0.39
E5	0.51	0.54	0.46	0.48
Instructor	0.52	0.53	0.45	0.46
EmbeddingGemma	0.49	0.49	0.42	0.41
ColBERT	0.58	0.60	0.45	0.42
Qwen3-Embedding	<u>0.61</u>	<u>0.65</u>	<u>0.50</u>	<u>0.49</u>
AgenticScholar	0.73	0.74	0.69	0.70

6 Experimental Study

We implement AgenticScholar [57] and conduct a comprehensive experimental study to evaluate its effectiveness, efficiency, and end-to-end usability. Our evaluation is grounded in a diverse corpus designed to assess the system’s versatility across various scholarly domains, covering broad domains, specialized sub-fields, and emerging areas. A complete list of topics is in Appendix F.

Our evaluation begins by benchmarking the performance of AgenticScholar’s core capabilities: retrieval (Section 6.1), information extraction and synthesis (Section 6.2), and knowledge discovery and generation (Section 6.3). To demonstrate its practical usability, we then present an end-to-end case study that simulates a realistic open-ended scholarly exploration workflow (Section 6.4). Finally, we conduct a series of ablation studies (Section 6.5) to validate the measurable benefits of our key architectural decisions.

6.1 Evaluation of Retrieval

We evaluate the effectiveness of AgenticScholar in paper retrieval (Tier 1 in Figure 3) using 237 research papers spanning 10 topics.

Query Generation. We construct 220 queries of two types: *factual queries*, which target explicit paper metadata (e.g., title, authors, venue) and research aspects (e.g., topics, problems, methods, datasets); and *analytical queries*, which require reasoning within or across topics. Each query is derived from paper metadata and aspect summaries and is paired with 1–10 manually verified ground-truth papers. Additional details are provided in Appendix H.1.

Baselines. We compare AgenticScholar with the classical lexical baseline *BM25* [68], strong dense retrievers – *GTR* [56], *E5* [84], *Instructor* [76], *EmbeddingGemma* [80], and *Qwen3-Embedding* [93] – and the late-interaction retriever *ColBERT* [70]. AgenticScholar and ColBERT have ~0.1B parameters, while others range from ~0.3B to ~1.5B. Most models use 768-d embeddings, except E5 (1024), and ColBERT uses 128-d token embeddings. All methods are reranked with GPT-4.1 on the top-*k* candidates for fair comparison [1]. We omit *LLM-Direct* retrieval due to context-length limitations.

Evaluation Metrics. We use R-Precision [10], which is precision at *R* (number of relevant papers), and Mean Average Precision (MAP) [18], which averages precision across relevant ranks. We omit Recall@*k* and Precision@*k* since fixed cutoffs are incomparable when queries have different numbers of relevant papers.

Results. Table 3 shows an overall result (see Appendix H.2 for results by topic). AgenticScholar consistently outperforms all baselines. This confirms its effectiveness, particularly in performing cross-modal synthesis that unifies precise lexical matching with the semantic generalization of dense encoders. For factual queries, this synthesis enables AgenticScholar to capture both explicit facts and semantically related expressions often missed by single-modal methods. For analytical queries,

Table 4. Effectiveness on Tier-2 queries.

Method	NDCG@1	NDCG@3	NDCG@5	NDCG@7
AgenticScholar	0.606	0.629	0.644	0.655
RAG	0.411	0.482	0.491	0.494

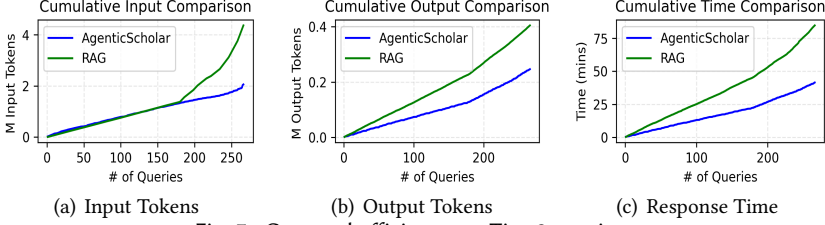


Fig. 7. Cost and efficiency on Tier-2 queries.

LLM-driven query decomposition and reranking facilitate nuanced reasoning over abstract research aspect, going beyond simple text similarity. These strengths demonstrate that AgenticScholar broadens retrieval coverage while enhancing contextual and analytical fidelity.

6.2 Information Extraction and Synthesis

We evaluate our system’s ability to extract relevant information and synthesize it into coherent answers (Tier 2 in Figure 3).

Query Generation. We consider four topics, each containing 10 papers, to construct two test sets.

- *LLM-generated test set.* Using three topics, we employ an LLM to generate evaluation queries. Based on our plan library (Appendix C), we generate 480 queries covering 16 single-document plans and 72 queries for six multi-document plans with group sizes ranging from 3 to 10 papers. In addition, we generate 12 queries with group sizes ranging from 3 to 10 papers that require dynamic plan generation.

- *Human-annotated test set.* We additionally construct a human-annotated test set consisting of 197 queries over two topics, derived from predefined single-document and multi-document plans that involve the Rank operator (see Table 12 in the appendix), enabling quantitative evaluation. Specifically, we manually examine the experimental results reported in each paper to determine rankings of methods or datasets. For single-document rank plans, methods or datasets are ranked according to commonly reported evaluation metrics within each paper. For multi-document rank plans, we group 3–10 papers and rank the evaluated methods based on shared datasets and metrics within each group.

Baselines. We compare AgenticScholar with an *RAG-based* method implemented using LangChain [38]. The method retrieves relevant document chunks and synthesizes answers using GPT-4.1 nano. We set the chunk size to 1000 to capture broader scholarly context and retrieve 8 chunks per document for each query.

Evaluation Metrics. We evaluate effectiveness on the human-annotated test set using $NDCG@k$ [34], a widely used ranking metric. Cost and efficiency are measured on the LLM-generated test set in terms of input/output tokens and response time.

Results. We observe the following:

Effectiveness. Table 4 reports $NDCG@k$ for k ranging from 1 to 7. AgenticScholar consistently outperforms RAG across all k values, indicating that it accurately extracts relevant information from papers and effectively executes the Rank operator to place correct results at the top of the ranked list.

Table 5. Results on Tier-3 queries. Elicit is the baseline; other entries report relative improvements (green, ↑) or degradations (red, ↓). *Efficiency* shows percentage runtime change (positive = faster).

Metric	Elicit	Gemini DR	Tongyi DR	Open DR	SmolAgent	AgenticScholar _{noR}	AgenticScholar _{noC}	AgenticScholar
<i>Research Trend Analysis — Problem-oriented</i>								
Correctness	5.84	+21.2%	-10.3%	+25.0%	+26.7%	+32.7%	+30.7%	+34.8%
Relevance	5.83	+45.5%	+41.2%	+56.1%	+47.5%	+52.1%	+53.5%	+54.7%
Diversity	5.42	+26.0%	+17.5%	+24.0%	+11.8%	+27.9%	+26.8%	+29.3%
Specificity	5.23	+54.7%	+25.4%	+53.7%	+23.1%	+59.1%	+62.9%	+61.8%
Efficiency	1,276 s	+40.2%	+68.1%	+71.6%	+18.7%	+68.9%	+70.5%	+69.6%
<i>Research Trend Analysis — Method-oriented</i>								
Correctness	6.02	+6.5%	-6.8%	+27.1%	+24.6%	+26.7%	+29.7%	+28.9%
Relevance	7.40	+13.9%	+2.7%	+21.6%	+6.1%	+18.4%	+15.4%	+20.1%
Diversity	4.83	+41.6%	+36.2%	+49.1%	+24.2%	+50.3%	+49.1%	+51.6%
Specificity	5.83	+21.3%	+16.6%	+37.9%	+17.5%	+41.2%	+36.0%	+41.9%
Efficiency	1,432 s	+52.9%	+69.7%	+73.8%	-47.2%	+75.1%	+71.8%	+74.2%
<i>Research Idea Exploration</i>								
Novelty	6.38	+17.1%	-21.6%	+16.0%	+12.7%	+22.7%	+26.0%	+25.1%
Feasibility	7.18	-5.0%	-1.3%	+2.4%	-2.5%	+6.5%	+6.5%	+8.4%
Impact	7.82	+0.3%	-10.9%	+1.0%	+5.5%	+3.2%	+3.2%	+5.8%
Efficiency	1,723 s	+66.7%	+77.3%	+80.7%	+91.9%	+75.3%	+77.4%	+78.0%
<i>Milestone Paper Selection</i>								
Impact	6.67	+19.9%	0.0%	+7.9%	-21.3%	+12.4%	+19.9%	+19.9%
Coverage	5.67	+20.5%	0.0%	+12.9%	-41.6%	+12.4%	+11.6%	+11.6%
Coherence	6.33	+23.7%	+5.4%	+16.9%	-40.8%	+10.5%	+15.7%	+13.1%
Efficiency	213 s	-209.9%	-36.6%	-48.4%	-333.8%	+99.6%	+99.6%	+99.6%

Cost and Efficiency. Figure 7 reports cumulative input tokens, output tokens, and response time for the *learned index* topic using the LLM-generated test set. Results on other topics show similar trends and are provided in Appendix I. Across all metrics, AgenticScholar consistently outperforms the standard RAG framework, using fewer input and output tokens and achieving lower cumulative latency as query volume increases.

6.3 Knowledge Discovery and Generation

We evaluate how effective AgenticScholar supports knowledge discovery and generation queries (Tier-3 in Figure 3), i.e., research trend analysis, idea exploration, and milestone paper selection. The implementation details are provided in Appendix E.

Query Generation. We use five research topics spanning broad domains, specialized sub-fields, and emerging areas (Appendix F). For each topic, we manually design Tier-3 queries of three types – *research trend analysis*, *research idea exploration*, and *milestone paper selection* – following a unified template that specifies the topic, query type, and output format. Query examples and prompt templates are provided in Appendix J.

Baselines. We compare AgenticScholar against three closed-source systems – Elicit [23], Tongyi Deep Research [67], and Gemini Deep Research [66] – and two open-source systems, SmolAgent [75] and Open Deep Researcher [60]. We also evaluate two variants of AgenticScholar: AgenticScholar_{noC}, which adopts LLM-generated reference taxonomies, and AgenticScholar_{noR}, which derives taxonomies solely from the corpus. For fairness, all open-source baselines and AgenticScholar use GPT-5 as the backbone LLM.

Evaluation Metrics. We use the *LLM-as-a-judge protocol* [28, 43, 65], with GPT-5 serving as an impartial reviewer to score each method on specific dimensions (1–10 scale).

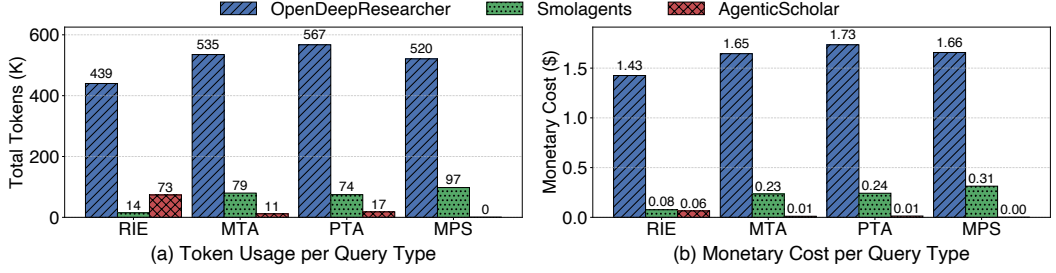


Fig. 8. The comparison of token and monetary cost on Tier-3 Queries (RIE: Research Idea Exploration; MTA: Method-oriented Trend Analysis; PTA: Problem-oriented Trend Analysis; MPS: Milestone Paper Selection).

Table 6. Breakdown of token cost on Tier-3 queries.

Query type	Planning ¹	Retrieval	Reasoning	Total Tokens
Research Idea Exploration	410	0	73,245	73,519
Method Trend Analysis	410	0	11,345	11,649
Problem Trend Analysis	410	0	17,463	17,935
Milestone Paper Selection	410	0	0	372

¹ As these queries directly match plans in our predefined plan library, their token cost in the planning stage remains fixed.

- **Research Trend Analysis:** (1) *Correctness* – factual accuracy, logical coherence, and temporal consistency; (2) *Relevance* – topical alignment with the target domain; (3) *Diversity* – coverage of distinct, non-overlapping trends; and (4) *Specificity* – concreteness and technical precision.
- **Research Idea Exploration:** (1) *Feasibility* – technical soundness and practical achievability; (2) *Novelty* – originality relative to existing research directions; and (3) *Impact* – potential significance for future studies.
- **Milestone Paper Selection:** (1) *Impact* – influence and contribution to the research community; (2) *Coverage* – representativeness across key subtopics; and (3) *Coherence* – consistency and thematic alignment within the selected set.

Additionally, for each query type, we report *efficiency*, measured as average inference time per query, as well as *token* and *monetary* costs for AgenticScholar and open-source baselines.

Results. We observe the following:

Effectiveness. As shown in Table 5, AgenticScholar achieves the best overall performance across nearly all dimensions.

- **Research trend analysis.** AgenticScholar and its variants perform strongly across all evaluation dimensions. High *correctness* and *relevance* stem from its knowledge-grounded reasoning pipeline, which decomposes topics into structured subtopics and retrieves evidence via semantically anchored nodes (e.g., Problem, Method). They also achieve superior *specificity* and *diversity*, reflecting the robustness of its taxonomy in modeling hierarchical problem–method relations. Notably, Open Deep Researcher attains comparable – and occasionally higher – *relevance*, which we attribute to its strong online search capability for dynamically retrieving up-to-date sources.
- **Research idea exploration.** Consistent with trend analysis, the AgenticScholar family produces research ideas that are *novel*, *feasible*, and *impactful*, driven by its structured discovery of underexplored problem–method intersections. This grounding enables forward-looking ideas while avoiding the speculative outputs common in baselines. Notably, AgenticScholar_{noR} achieves slightly higher *Novelty*, suggesting that removing reference-guided constraints reduces anchoring bias and

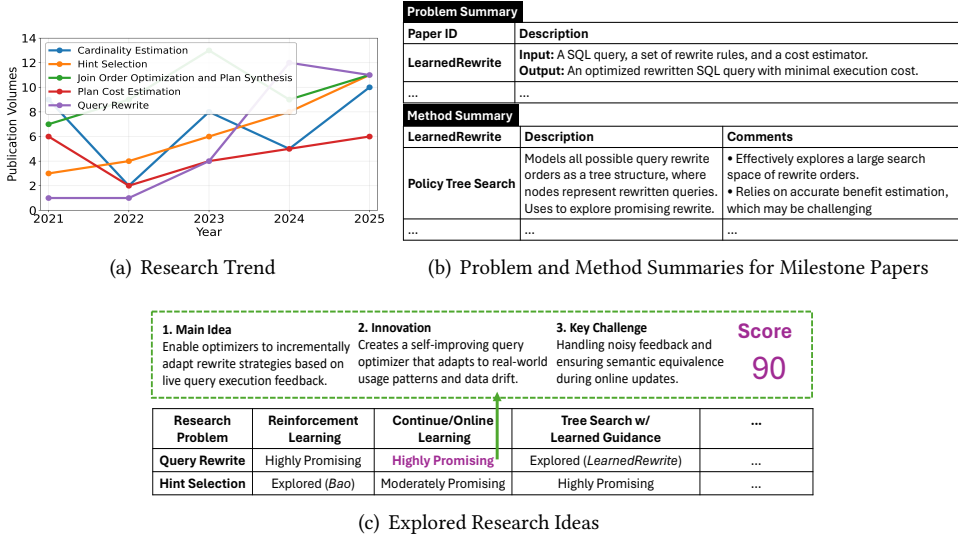


Fig. 9. AgenticScholar's results for case study on learning-based query optimization.

encourages broader hypothesis generation. However, this increased exploration comes at the cost of lower *Impact* and *Feasibility*.

- **Milestone paper selection.** AgenticScholar and its variants remain highly competitive, achieving strong *Impact* scores. Their relatively lower *Coverage* and *Coherence* compared to Gemini Deep Research and Open Deep Researcher. This may stem from the latter's web search capabilities, which enable broader and more up-to-date evidence retrieval.

Efficiency. AgenticScholar consistently delivers strong efficiency gains across Tier-3 tasks, with substantial speedups on *Milestone Paper Selection*. The only exception is *Research Idea Exploration*, where it is slightly slower than Open Deep Researcher, likely due to evaluating a larger space of problem–method combinations for verification and scoring. Overall, these results indicate that structure-guided planning and targeted retrieval reduce unnecessary browsing and stabilize tool-call overhead, with the largest benefits emerging in focused selection tasks.

Token and Monetary Costs. Figure 8 compares token and monetary costs against open-source baselines, with all systems using a unified GPT-5 backbone. Owing to its internal knowledge layer, AgenticScholar minimizes token consumption across most query types, except research idea exploration. This efficiency stems from its lightweight LLM-based planner and reliance on internal knowledge for search and reasoning, which substantially reduces token overhead in the planning and retrieval stage. For instance, the average token usage for Open Deep Researcher and SmolAgent during planning and retrieval is 237,057 and 12,056, respectively, whereas AgenticScholar consumes only 355 tokens on average. Table 6 further provides a stage-level breakdown, AgenticScholar consistently maintains minimal token consumption in planning and retrieval

6.4 Case Study on Learning-based Query Optimization

To illustrate the end-to-end capabilities of AgenticScholar in a realistic research setting, we present a case study simulating a typical researcher's workflow. The task is to explore the rapidly evolving area of *learning-based query optimization* and identify underexplored research directions – an objective that challenges existing systems due to multi-step semantic reasoning over large scholarly corpora. The case study proceeds as a four-step progressive query session, demonstrating how AgenticScholar enables iterative intent refinement and reasoning toward novel research ideas.

Step 1: Understanding the Research Landscape (Tier 3). The researcher begins with an open-ended question to gain a high-level overview of the field and issues the query: *Q1: “What are the overall research trends in applying learning-based methods to query optimization since 2021?”* As shown in Figure 9(a), AgenticScholar generates a trend chart depicting publication volumes over time across key sub-topics. This goes beyond simple keyword search – AgenticScholar’s *Knowledge Representation Layer* has already organized the corpus into a hierarchical taxonomy of research problems in query optimization. Its *Execution Layer* then applies *Traverse* and *Aggregate* operators to quantify and visualize the evolution of these research focus areas, allowing users to understand how interest shifts across sub-topics.

Step 2: Identifying Foundational Studies Across Key Sub-fields (Tier 3). The trend analysis from Step 1 provides a crucial insight: a clear upward trend in *query rewrite* and *hint selection*, indicating that the community is actively exploring these techniques to build more robust query plans under estimation errors. To further understand this development, the researcher decides to investigate foundational studies across these two areas and issue the query *“Can you identify three milestone papers for ‘query rewrite’ and ‘hint selection’?”* AgenticScholar returns the three most influential papers for each of the two requested topics: *‘learned query rewriting’*: SIA [94], LLM-R² [45], and LearnedRewrite [95]; *‘optimizer hint selection’*: Bao [51], Steering [55], and COOL [89]. This highlights AgenticScholar’s ability to conduct fine-grained, taxonomy-guided knowledge discovery that identifies representative contributions within specific sub-fields.

Step 3: Summarizing Core Problems and Methods (Tier 2). The researcher seeks a concise overview of each group of milestone papers to understand the central problems and methodological paradigms. The researcher issues the query: *“Could you summarize the core problems and methods for each of these three groups of papers?”* As shown in Figure 9(b), AgenticScholar produces structured summaries for each paper (e.g., LearnedRewrite), distilling both the problem formulation and proposed method. This step demonstrates how AgenticScholar’s *Extract* and *Summarize* operators transform unstructured scholarly text into interpretable, structured insights – laying the foundation for higher-level reasoning.

Step 4: Identifying Novel Research Opportunities (Tier 3). With a clear understanding of the milestone papers’ problems and methods, the researcher is now ready to identify new research directions and issues the query: *“Okay, what would be some promising research ideas or directions to explore in query rewriting and hint selection?”* As shown in Figure 9(c), AgenticScholar traverses its knowledge graph to generate problem–method combinations representing underexplored. It then ranks these ideas to provide concise descriptions of the most promising ideas for future research.

6.5 Validating Architectural Design

6.5.1 The Effectiveness of Reference-enhanced Taxonomy Construction. To assess the quality of taxonomies produced by our reference-enhanced construction method, we follow [96] and build a benchmark using gold-standard taxonomies manually curated from seven authoritative surveys published in ACM Computing Surveys¹ (see Appendix K.1.1 for details). As baselines, we include two state-of-the-art taxonomy construction methods, *TaxoAdapt* [36] and *CHTaxo* [96], along with two variants of our approach, AgenticScholar_{noR} and AgenticScholar_{noC}. We evaluate taxonomy quality using *soft* Precision, Recall, and F1 [25], which account for semantic similarity between concept labels and partial structural alignment, and further assess structural quality via an LLM-as-judge protocol (See Appendix K.1.2 for details). As shown in Table 7, AgenticScholar consistently achieves the best overall performance across *soft* Precision, Recall, and F1 at nearly all taxonomy levels. These results indicate improvements in both concept identification (node quality) and

¹<https://dl.acm.org/journal/csur>

Table 7. Level-wise taxonomy evaluation.

Level	Methods	Node quality (soft)			Edge quality (soft)		
		Pre.	Rec.	F1.	Pre.	Rec.	F1.
1	TaxoAdapt	0.632	0.561	0.594	0.498	0.421	0.456
	CHTaxo	0.661	0.598	0.628	0.523	0.462	0.491
	AgenticScholar _{noR}	0.694	0.640	0.677	0.562	0.504	0.531
	AgenticScholar _{noC}	0.688	0.676	0.682	0.586	0.512	0.547
	AgenticScholar	0.718	0.684	0.703	0.574	0.538	0.555
2	TaxoAdapt	0.598	0.532	0.563	0.458	0.392	0.422
	CHTaxo	0.620	0.558	0.587	0.487	0.424	0.453
	AgenticScholar _{noR}	0.673	0.587	0.628	0.515	0.456	0.484
	AgenticScholar _{noC}	0.649	0.632	0.640	0.541	0.466	0.501
	AgenticScholar	0.684	0.642	0.657	0.549	0.487	0.507
3	TaxoAdapt	0.543	0.472	0.505	0.402	0.327	0.361
	CHTaxo	0.568	0.495	0.529	0.428	0.345	0.382
	AgenticScholar _{noR}	0.621	0.533	0.574	0.452	0.396	0.422
	AgenticScholar _{noC}	0.593	0.574	0.583	0.477	0.406	0.439
	AgenticScholar	0.608	0.582	0.595	0.466	0.432	0.448

Table 8. Effectiveness of Predefined Plan Selection.

Variant	Input Tokens (K)	Output Tokens (K)	Planning Time (s)
Enable	0.41	0.013	0.96
Disable	4.08	0.77	15.17

Table 9. Robustness through Self-Correction.

	0	1	2	3
Frequency	10	4	1	1
Cum. Pct.	62.5%	87.5%	93.75%	100%

hierarchical organization (edge quality), showing that anchoring taxonomies in corpus evidence while leveraging LLM knowledge yields more coherent and fine-grained structures.

6.5.2 The Effectiveness of Hybrid Planning.

Efficiency of Predefined Plan Selection We generate 50 Tier-2 queries which can hit our predefined plans pool on the *vector search* topic and compare two variants: enable predefined plan selection (Enable) and disable it (Disable). Table 8 shows the average *number of input/output tokens* and *planning latency* per query of each variant. We can observe: with predefined plan selection enabled, AgenticScholar reduces both input/output tokens and planning time by more than 90%, indicating that Predefined Plan Selection can effectively reduce token overhead and planning latency.

Robustness through Self-Correction We designed 16 complex Tier-2 queries on the *vector search* topic that genuinely reflect our daily research needs. We record the frequency and cumulative percentage of different repair attempts in Table 9. We observe that in most cases ($\geq 60\%$), AgenticScholar is already able to generate perfect plans, and with a reasonable small self-correction overhead (≤ 3 times).

6.5.3 The Impact of Execution Performance Optimizations. We generated 266 Tier-2 queries on the *learned indexes* topic and compare three variants: disable cache (W/O Cache), disable cache and parallel processing (W/O Cache and Para. Proc.), and enable all of them (Enable All). Table 10 shows the average *number of input/output tokens* and *latency* per query of each variant. We can

Table 10. Impact of Performance Optimizations.

Variant	Input Tokens (K)	Output Tokens (K)	Time (s)
Enable All	7.7	0.9	9.3
W/O Cache	23.0	2.8	13.7
W/O Cache and Para. Proc.	22.8	2.8	23.3

Table 11. The impact of backbone LLMs for research idea exploration. The colors express relative values (worse than average, average, and better than average).

Metric	Token cost	Monetary cost	Efficiency	Novelty	Feasibility	Impact
GPT-5	73,245	\$0.640	379.1s	8.22	7.92	8.42
GPT-5 Mini	66,531	\$0.116	338.1s	7.60	7.36	7.66
Claude 4.5 Sonnet	71,017	\$0.957	468.9s	7.98	7.66	8.02
Claude 4.5 Opus	77,001	\$1.737	673.5s	8.12	7.78	8.18
Gemini-3.0 Flash	60,240	\$0.160	305.2s	7.72	7.44	7.78
Gemini-3.0 Pro	74,094	\$0.793	433.7s	8.36	7.84	8.34
Grok-4.1 Fast (NR)	54,429	\$0.026	147.4s	7.28	7.14	7.34
Grok-4.1 Fast (R)	93,794	\$0.046	233.6s	7.38	7.22	7.44
Average	71,294	\$0.559	372.4s	7.83	7.55	7.90
Standard deviation	±16.6%	±107.2%	±43.0%	±5.1%	±4.0%	±5.2%

observe: with caching and parallel processing enabled, AgenticScholar uses about one-third of the input tokens and achieves the lowest latency (9.3 s). Disabling caching increases token usage and latency, while removing both optimizations roughly doubles the runtime.

6.5.4 Impact of Backbone LLMs on End-to-End Performance. We study how different backbone LLMs affect the end-to-end performance of AgenticScholar. We instantiate AgenticScholar with eight representative backbones from different providers, including GPT-5 and GPT-5 mini [58, 59], Claude Sonnet 4.5 and Claude Opus 4.5 [3, 4], Gemini 3 Flash Preview and Gemini 3 Pro Preview [16, 17], and Grok-4.1 Fast variants without and with explicit reasoning (NR/R) [87]. For fair comparison, we fix the agentic pipeline, prompts, and all non-LLM components, varying only the backbone. Table 11 reports results for research idea exploration (see Appendix K.3 for research trend analysis). Following [39], we categorize backbone LLMs into Better, Average, and Worse based on relative performance (top, middle, and bottom 33%). We observe that: (1) backbone choice has limited impact on effectiveness but substantially influences efficiency and cost; (2) GPT-5 achieves the strongest effectiveness, followed by Gemini-3.0-Pro; (3) Grok-4.1 Fast variants deliver the best efficiency and lowest cost; and (4) among the evaluated backbones, Gemini-3.0-Flash offers the best accuracy–efficiency trade-off.

7 Lessons Learned and Future Work

Knowledge Graph Construction and Maintenance. Our experience shows that the quality of structured knowledge is pivotal to the success of AgenticScholar. While the automated pipeline for constructing knowledge graphs and taxonomies is effective, it remains prone to occasional errors in entity extraction, relation detection, and semantic labeling that can propagate through downstream components. Heavy reliance on large models also incurs substantial computational and financial cost. Future efforts can be made to incorporate human-in-the-loop validation and cost-efficient hybrid pipelines that combine lightweight NLP components with compact fine-tuned models.

Multi-Modal Data Extraction. Figures in scholarly documents are essential for high-quality scholarly synthesis, yet quantitative extraction remains a challenge. Recovering accurate numerical values from complex plots, such as spider charts or line charts, remains unreliable, even with

SOTA multi-modal LLMs. This insight motivates the integration of specialized vision-language models and structured figure-parsing frameworks to improve numerical fidelity and support richer evidence-grounded analysis.

Model Selection and Monetary Cost in Complex Analysis. A key lesson is that scholarly queries often require reasoning across multiple papers and synthesizing experimental evidence. This creates an important trade-off among model efficiency, reliability, and cost-effectiveness. Lightweight models such as GPT-4 . 1-nano offer faster inference and lower monetary cost, making them attractive for cost-sensitive large-scale use, but they are also more prone to hallucinations than stronger models such as GPT-5 or Gemini 2.5 Pro. Future work could therefore explore adaptive model selection strategies that dynamically choose models based on task complexity and the desired cost-quality balance. One promising direction is to decompose complex analysis into smaller subtasks that can be handled by lightweight, more cost-effective models, while reserving more powerful and expensive models for reasoning-intensive stages. Such a design could improve overall cost-effectiveness while maintaining a strong balance among accuracy, speed, and monetary cost.

Domain-Aware Planning and Reasoning. Finally, our study underscores the value of domain sensitivity in agentic reasoning. While AgenticScholar already generalizes across disciplines, domain-agnostic planning can miss field-specific reasoning conventions – such as performance-driven evaluation in computer science, proof-centric validation in physics, or evidence hierarchies in medicine. Future efforts will explore domain-aware planning that integrates domain-specific reasoning and analytical workflows, enabling results that are both precise and contextually aligned with expert reasoning practices.

8 Conclusion

In this paper, we have developed AgenticScholar, an agentic data management system for scholarly corpora. It unifies knowledge representation, hybrid query planning, and operator-based execution to support diverse retrieval, synthesis, and discovery tasks. To the best of our knowledge and evaluation, AgenticScholar is the first system to execute agentic reasoning over multi-modal scholarly data through DAG-based plans, offering a practical foundation for future research in agentic data management. While AgenticScholar demonstrates strong performance on several scholarly domains, our current evaluation focuses primarily on research papers from a limited set of fields in computer science. Supporting broader disciplines may require domain-aware planning and reasoning mechanisms to better capture field-specific structures and analytical conventions.

References

- [1] Anirudh Ajith, Mengzhou Xia, Alexis Chevalier, Tanya Goyal, Danqi Chen, and Tianyu Gao. 2024. Litsearch: A retrieval benchmark for scientific literature search. *arXiv preprint arXiv:2407.18940* (2024).
- [2] Günes Aluç, David DeHaan, and Ivan T. Bowman. 2012. Parametric Plan Caching Using Density-Based Clustering. In *ICDE*. 402–413.
- [3] Anthropic. 2025. Introducing Claude Sonnet 4.5. <https://www.anthropic.com/news/claude-sonnet-4-5>. Accessed: 2026-01-02.
- [4] Anthropic. 2026. Claude Opus 4.5. <https://www.anthropic.com/claude/opus>. Accessed: 2026-01-02.
- [5] Llama API. 2025. <https://www.llama.com/products/llama-api/>.
- [6] Mistral API. 2025. <https://docs.mistral.ai/capabilities/document/>.
- [7] OpenAI API. 2025. <https://platform.openai.com/docs/models>.
- [8] Qwen API. 2025. <https://qwen.ai/apiplatform>.
- [9] Semantic Scholar API. 2025. <https://api.semanticscholar.org/>
- [10] Javed A Aslam and Emine Yilmaz. 2005. A geometric interpretation and analysis of r-precision. In *Proceedings of the 14th ACM international conference on Information and knowledge management*. 664–671.
- [11] Ai2 Asta. 2025. <https://asta.allen.ai/chat>
- [12] Chengliang Chai, Jiajun Li, Yuhao Deng, Yuanhao Zhong, Ye Yuan, Guoren Wang, and Lei Cao. 2025. Doctopus: Budget-aware Structural Table Extraction from Unstructured Documents. *Proc. VLDB Endow.* 18, 11 (2025), 3695–3707.
- [13] ChatPDF. 2025. <https://www.chatpdf.com/>
- [14] Boqi Chen, Fandi Yi, and Dániel Varró. 2023. Prompting or Fine-tuning? A Comparative Study of Large Language Models for Taxonomy Construction. *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)* (2023), 588–596.
- [15] Jindong Chen, Ao Wang, Jiangjie Chen, Yanghua Xiao, Zhendong Chu, Jingping Liu, Jiaqing Liang, and Wei Wang. 2019. CN-Probase: A Data-Driven Approach for Large-Scale Chinese Taxonomy Construction. *2019 IEEE 35th International Conference on Data Engineering (ICDE)* (2019), 1706–1709.
- [16] Google Cloud. 2025. Gemini 3 Flash (Preview) on Vertex AI. <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/3-flash>. Accessed: 2026-01-02.
- [17] Google Cloud. 2025. Gemini 3 Pro Preview (Vertex AI Model Garden). <https://console.cloud.google.com/vertex-ai/publishers/google/model-garden/gemini-3-pro-preview>. Accessed: 2026-01-02.
- [18] Gordon V Cormack and Thomas R Lynam. 2006. Statistical precision of information retrieval evaluation. In *Proceedings of the 29th annual international ACM SIGIR conference on Research and development in information retrieval*. 533–540.
- [19] Neo4j Graph Database. 2025. <https://neo4j.com/>
- [20] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). *arXiv:2401.08281 [cs.LG]*
- [21] Kayhan Dursun, Carsten Binnig, Ugur Çetintemel, and Tim Kraska. 2017. Revisiting Reuse in Main Memory Database Systems. In *SIGMOD Conference*. ACM, 1275–1289.
- [22] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitan-sky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).
- [23] Elicit. 2025. <https://elicit.com/>
- [24] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. 2018. Cypher: An Evolving Query Language for Property Graphs. In *SIGMOD*. 1433–1445.
- [25] Pasi Fränti and Radu Marinescu-Istodor. 2023. Soft precision and recall. *Pattern Recognition Letters* 167 (2023), 115–121.
- [26] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *CoRR abs/2312.10997* (2023). <https://doi.org/10.48550/arXiv.2312.10997>
- [27] Gemini. 2025. <https://ai.google.dev/gemini-api/docs>.
- [28] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. 2024. A survey on llm-as-a-judge. *arXiv preprint arXiv:2411.15594* (2024).
- [29] Dong Guo, Faming Wu, Feida Zhu, Fuxing Leng, Guang Shi, Haobin Chen, Haoqi Fan, Jian Wang, Jianyu Jiang, Jiawei Wang, et al. 2025. Seed1. 5-v1 technical report. *arXiv preprint arXiv:2505.07062* (2025).
- [30] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2024. LightRAG: Simple and Fast Retrieval-Augmented Generation. *CoRR abs/2410.05779* (2024).
- [31] Yichen He, Guanhua Huang, Peiyuan Feng, Yuan Lin, Yuchen Zhang, Hang Li, and Weinan E. 2025. PaSa: An LLM Agent for Comprehensive Academic Paper Search. In *ACL (1)*. Association for Computational Linguistics, 11663–11679.

- [32] Jiaxin Huang, Yiqing Xie, Yu Meng, Yunyi Zhang, and Jiawei Han. 2020. CoRel: Seed-Guided Topical Taxonomy Construction by Concept Learning and Relation Transferring. *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (2020), 1928–1936.
- [33] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).
- [34] Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated gain-based evaluation of IR techniques. *ACM Trans. Inf. Syst.* 20, 4 (2002), 422–446.
- [35] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active Retrieval Augmented Generation. In *EMNLP*. 7969–7992.
- [36] Priyanka Kargupta, Nan Zhang, Yunyi Zhang, Rui Zhang, Prasenjit Mitra, and Jiawei Han. 2025. TaxoAdapt: Aligning LLM-Based Multidimensional Taxonomy Construction to Evolving Research Corpora. *arXiv preprint arXiv:2506.10737* (2025).
- [37] Uri Katz, Mosh Levy, and Yoav Goldberg. 2024. Knowledge Navigator: LLM-guided Browsing Framework for Exploratory Search in Scientific Literature. In *Findings of EMNLP*. 8838–8855.
- [38] LangChain. 2025. <https://www.langchain.com/>.
- [39] Jiale Lao, Andreas Zimmerer, Olga Ovcharenko, Tianji Cong, Matthew Russo, Gerardo Vitagliano, Michael Cochez, Fatma Özcan, Gautam Gupta, Thibaud Hottelier, H. V. Jagadish, Kris Kissel, Sebastian Schelter, Andreas Kipf, and Immanuel Trummer. 2025. SemBench: A Benchmark for Semantic Query Processing Engines. *CoRR* abs/2511.01716 (2025). <https://doi.org/10.48550/arXiv.2511.01716>
- [40] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter A. Boncz, Alfons Kemper, and Thomas Neumann. 2015. How Good Are Query Optimizers, Really? *Proc. VLDB Endow.* 9, 3 (2015), 204–215.
- [41] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [42] Guoliang Li, Xuanhe Zhou, Ji Sun, Xiang Yu, Yue Han, Lianyan Jin, Wenbo Li, Tianqing Wang, and Shifu Li. 2021. openGauss: An Autonomous Database System. *Proc. VLDB Endow.* 14, 12 (2021), 3028–3041.
- [43] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024. Llm-as-judges: a comprehensive survey on llm-based evaluation methods. *arXiv preprint arXiv:2412.05579* (2024).
- [44] Yumeng Li, Guang Yang, Hao Liu, Bowen Wang, and Colin Zhang. 2025. dots.ocr: Multilingual Document Layout Parsing in a Single Vision-Language Model. *arXiv:2512.02498 [cs.CV]* <https://arxiv.org/abs/2512.02498>
- [45] Zhaodonghui Li, Haitao Yuan, Huiming Wang, Gao Cong, and Lidong Bing. 2024. LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System for Boosting Query Efficiency. *Proc. VLDB Endow.* 18, 1 (2024), 53–65.
- [46] Yiming Lin, Madelon Hulsebos, Ruiying Ma, Shreya Shankar, Sepanta Zeighami, Aditya G. Parameswaran, and Eugene Wu. 2024. Towards Accurate and Efficient Document Analytics with Large Language Models. *CoRR* abs/2405.04674 (2024).
- [47] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, et al. 2025. Palimpsest: Optimizing ai-powered analytics with declarative query processing. In *CIDR*. 2.
- [48] Kyle Lo, Lucy Lu Wang, Mark Neumann, Rodney Kinney, and Daniel S. Weld. 2020. S2ORC: The Semantic Scholar Open Research Corpus. In *ACL*. 4969–4983.
- [49] Yuyin Lu, Hegang Chen, Pengbo Mao, Yanghui Rao, Haoran Xie, Fu Lee Wang, and Qing Li. 2024. Self-supervised Topic Taxonomy Discovery in the Box Embedding Space. *Trans. Assoc. Comput. Linguistics* 12 (2024), 1401–1416.
- [50] Haoran Luo, Guanting Chen, Yandan Zheng, Xiaobao Wu, Yikai Guo, Qika Lin, Yu Feng, Zemin Kuang, Meina Song, Yifan Zhu, et al. 2025. HyperGraphRAG: Retrieval-Augmented Generation with Hypergraph-Structured Knowledge Representation. *arXiv preprint arXiv:2503.21322* (2025).
- [51] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *SIGMOD 2021*. ACM, 1275–1288.
- [52] Mineru. April, 2025. https://mineru.readthedocs.io/en/latest/user_guide/usage/api.html.
- [53] Songsong Mo, Yile Chen, Hao Wang, Gao Cong, and Zhifeng Bao. 2023. Lemo: A Cache-Enhanced Learned Optimizer for Concurrent Queries. *Proc. ACM Manag. Data* 1, 4 (2023), 247:1–247:26.
- [54] MySQL. 2025. <https://www.mysql.com/>.
- [55] Parimarjan Negi, Matteo Interlandi, Ryan Marcus, Mohammad Alizadeh, Tim Kraska, Marc T. Friedman, and Alekh Jindal. 2021. Steering Query Optimizers: A Practical Take on Big Data Workloads. In *SIGMOD 2021*. ACM, 2557–2569.
- [56] Jianmo Ni, Chen Qu, Jing Lu, Zhuyun Dai, Gustavo Hernandez Abrego, Ji Ma, Vincent Y Zhao, Yi Luan, Keith B Hall, Ming-Wei Chang, et al. 2021. Large dual encoders are generalizable retrievers. *arXiv preprint arXiv:2112.07899* (2021).
- [57] Source Code of AgenticScholar. 2026. <https://github.com/DataAutonomyLab/agentic scholar>.

- [58] OpenAI. 2026. GPT-5 mini Model (OpenAI API Documentation). <https://platform.openai.com/docs/models/gpt-5-mini>. Accessed: 2026-01-02.
- [59] OpenAI. 2026. GPT-5 Model (OpenAI API Documentation). <https://platform.openai.com/docs/models/gpt-5>. Accessed: 2026-01-02.
- [60] OpenDeepResearcher. 2025. <https://github.com/mshumer/OpenDeepResearcher>.
- [61] Linke Ouyang, Yuan Qu, Hongbin Zhou, Jiawei Zhu, Rui Zhang, Qunshu Lin, Bin Wang, Zhiyuan Zhao, Man Jiang, Xiaomeng Zhao, Jin Shi, Fan Wu, Pei Chu, Minghao Liu, Zhenxiang Li, Chao Xu, Bo Zhang, Botian Shi, Zhongying Tu, and Conghui He. 2025. OmniDocBench: Benchmarking Diverse PDF Document Parsing with Comprehensive Annotations. In *CVPR*. Computer Vision Foundation / IEEE, 24838–24848.
- [62] Liana Patel, Siddharth Jha, Melissa Z. Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Towards AI-Based Data Analytics with Accuracy Guarantees. *Proc. VLDB Endow.* 18, 11 (2025), 4171–4184.
- [63] PostgreSQL. 2025. <https://www.postgresql.org/>.
- [64] Jason Priem, Heather A. Piwowar, and Richard Orr. 2022. OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. *CoRR* abs/2205.01833 (2022). <https://doi.org/10.48550/arXiv.2205.01833>
- [65] Vyas Raina, Adian Liusie, and Mark Gales. 2024. Is LLM-as-a-Judge Robust? Investigating Universal Adversarial Attacks on Zero-shot LLM Assessment. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 7499–7517.
- [66] Gemini Deep Research. 2025. <https://gemini.google/overview/deep-research/>.
- [67] Tongyi Deep Research. 2025. <https://github.com/Alibaba-NLP/DeepResearch>.
- [68] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.
- [69] Matthew Russo, Sivaprasad Sudhir, Gerardo Vitagliano, Chunwei Liu, Tim Kraska, Samuel Madden, and Michael J. Cafarella. 2025. Abacus: A Cost-Based Optimizer for Semantic Operator Systems. *CoRR* abs/2505.14661 (2025).
- [70] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2022. ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. In *NAACL-HLT*. Association for Computational Linguistics, 3715–3734.
- [71] Patricia G. Selinger, Morton M. Astrahan, Donald D. Chamberlin, Raymond A. Lorie, and Thomas G. Price. 1979. Access Path Selection in a Relational Database Management System. In *SIGMOD*. 23–34.
- [72] Chao Shang, Sarthak Dash, Md. Faisal Mahbub Chowdhury, Nandana Mihindukulasooriya, and A. Gliozzo. 2020. Taxonomy Construction of Unseen Domains via Graph-based Cross-Domain Knowledge Transfer. In *Annual Meeting of the Association for Computational Linguistics*. 2701–2709.
- [73] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G. Parameswaran, and Eugene Wu. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *Proc. VLDB Endow.* 18, 9 (2025), 3035–3048.
- [74] Jiaming Shen, Zeqiu Wu, Dongming Lei, Chao Zhang, Xiang Ren, Michelle T. Vanni, Brian M. Sadler, and Jiawei Han. 2018. HiExpan: Task-Guided Taxonomy Construction by Hierarchical Tree Expansion. *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (2018), 2180–2189.
- [75] Smolagents. 2025. <https://smolagents.org/>.
- [76] Hongjin Su, Weijia Shi, Jungo Kasai, Yizhong Wang, Yushi Hu, Mari Ostendorf, Wen-tau Yih, Noah A Smith, Luke Zettlemoyer, and Tao Yu. 2022. One embedder, any task: Instruction-finetuned text embeddings. *arXiv preprint arXiv:2212.09741* (2022).
- [77] Ji Sun, Guoliang Li, Peiyao Zhou, Yihui Ma, Jingzhe Xu, and Yuan Li. 2025. AgenticData: An Agentic Data Analytics System for Heterogeneous Data. *CoRR* abs/2508.05002 (2025).
- [78] Yushi Sun, Hao Xin, Kai Sun, Yifan Ethan Xu, Xiao Yang, Xin Luna Dong, Nan Tang, and Lei Chen. 2024. Are Large Language Models a Good Replacement of Taxonomies? *arXiv preprint arXiv:2406.11131* (2024).
- [79] Zhaoze Sun, Chengliang Chai, Qiyan Deng, Kaisen Jin, Xinyu Guo, Han Han, Ye Yuan, Guoren Wang, and Lei Cao. 2025. QUEST: Query Optimization in Unstructured Document Analysis. *Proc. VLDB Endow.* 18, 11 (2025), 4560–4573.
- [80] Henrique Schechter Vera, Sahil Dua, Biao Zhang, Daniel Salz, Ryan Mullins, Sindhu Raghuram Panyam, Sara Smoot, Iftekhar Naim, Joe Zou, Feiyang Chen, et al. 2025. Embeddinggemma: Powerful and lightweight text representations. *arXiv preprint arXiv:2509.20354* (2025).
- [81] Huaiyu Wan, Yutao Zhang, Jing Zhang, and Jie Tang. 2019. AMiner: Search and Mining of Academic Social Networks. *Data Intell.* 1, 1 (2019), 58–76.
- [82] Jiayi Wang and Jianhua Feng. 2025. Unify: An Unstructured Data Analytics System. In *ICDE*. 4662–4674.
- [83] Kuansan Wang, Zhihong Shen, Chiyuan Huang, Chieh-Han Wu, Yuxiao Dong, and Anshul Kanakia. 2020. Microsoft Academic Graph: When experts are not enough. *Quant. Sci. Stud.* 1, 1 (2020), 396–413.
- [84] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533* (2022).

- [85] Yujing Wang, Hainan Zhang, Liang Pang, Binghui Guo, Hongwei Zheng, and Zhiming Zheng. 2025. MaFeRw: Query Rewriting with Multi-Aspect Feedbacks for Retrieval-Augmented Large Language Models. In *AAAI*. 25434–25442.
- [86] Haoran Wei, Chenglong Liu, Jinyue Chen, Jia Wang, Lingyu Kong, Yanming Xu, Zheng Ge, Liang Zhao, Jianjian Sun, Yuang Peng, Chunrui Han, and Xiangyu Zhang. 2024. General OCR Theory: Towards OCR-2.0 via a Unified End-to-end Model. *CoRR* abs/2409.01704 (2024).
- [87] xAI. 2025. Grok 4.1 Fast and Agent Tools API. <https://x.ai/news/grok-4-1-fast>. Accessed: 2026-01-02.
- [88] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. *CoRR* abs/2309.07597 (2023).
- [89] Xianghong Xu, Zhibing Zhao, Tieying Zhang, Rong Kang, Luming Sun, and Jianjun Chen. 2023. COOOL: A Learning-To-Rank Approach for SQL Hint Recommendations. In *VLDB 2023 (CEUR Workshop Proceedings, Vol. 3462)*.
- [90] Qingkai Zeng, Yuyang Bai, Zhaoxuan Tan, Shangbin Feng, Zhenwen Liang, Zhihan Zhang, and Meng Jiang. 2024. Chain-of-Layer: Iteratively Prompting Large Language Models for Taxonomy Induction from Limited Examples. *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (2024)*, 3093–3102.
- [91] Chao Zhang, Fangbo Tao, Xiusi Chen, Jiaming Shen, Meng Jiang, Brian M. Sadler, Michelle T. Vanni, and Jiawei Han. 2018. TaxoGen: Unsupervised Topic Taxonomy Construction by Adaptive Term Embedding and Clustering. *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (2018)*.
- [92] Jintao Zhang, Guoliang Li, and Jinyang Su. 2025. SAGE: A Framework of Precise Retrieval for RAG. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 1388–1401.
- [93] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *CoRR* abs/2506.05176 (2025).
- [94] Qi Zhou, Joy Arulraj, Shamkant B. Navathe, William Harris, and Jinpeng Wu. 2021. SIA: Optimizing Queries using Learned Predicates. In *SIGMOD 2021*, Guoliang Li, Zhanhuai Li, Stratos Idreos, and Divesh Srivastava (Eds.). ACM, 2169–2181.
- [95] Xuanhe Zhou, Guoliang Li, Chengliang Chai, and Jianhua Feng. 2021. A Learned Query Rewrite System using Monte Carlo Tree Search. *Proc. VLDB Endow.* 15, 1 (2021), 46–58.
- [96] Kun Zhu, Lizi Liao, Yuxuan Gu, Lei Huang, Xiaocheng Feng, and Bing Qin. 2025. Context-Aware Hierarchical Taxonomy Generation for Scientific Papers via LLM-Guided Multi-Aspect Clustering. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 15627–15645.

Appendix

A Demonstration of AgenticScholar

AgenticScholar supports a spectrum of scholarly query intents within a unified, interactive workflow. In the following, we will demonstrate how a researcher can issue queries at multiple tiers and engage with the system’s interfaces.

Scenario. Consider a database researcher exploring the topic “AI4DB”. (1) *Information Extraction and Synthesis.* The researcher aims to determine whether the paper FLAT’s experimental results substantiate the claims made in its introduction. As illustrated in Figure 10, the researcher opens a new session via + New Chat, submits the query “Do the experimental results in these papers support the claims made in their introductions?”, and AgenticScholar returns a structured, evidence-backed answer.

(2) *Knowledge Discovery and Generation.* The researcher moves from single-paper analysis to corpus-level exploration of a recently collected “AI4DB” paper set, proceeding through three queries, such as research trend analysis, research idea exploration, and milestone paper selection:

- *Research trend analysis.* As shown in Figure 11, the left panel displays the hierarchical tree view of the problem taxonomy. Users can select one or more nodes within the taxonomy for trend analysis. The right panel then searches relevant papers online for these selected topics and visualizes their publication trends from 2020 to 2025 as line charts. When a user clicks on a specific subtopic represented by a line in the chart—e.g., *learned index structures*—a descriptive panel appears in the lower-right corner of AgenticScholar, providing a concise definition of the subtopic and highlighting representative works.

- *Research idea exploration.* The researcher then requests research idea exploration. AgenticScholar analyzes the corpus against problem and method taxonomies, aligns papers to these categories, and constructs a matrix whose rows are problem categories and columns are method categories (Figure 12). Each cell displays the number of matched papers. Interacting with a sparse (or empty) cell—e.g., the problem *Cardinality and Selectivity Estimation* crossed with the method *Learned and Autonomous DB Architectures*—opens a right-side panel that synthesizes the gap and surfaces promising research directions.

- *Milestone paper selection.* Finally, the researcher issues a milestone identification request. As shown in Figure 13, they choose relevant subtopics from the problem taxonomy on the left, and configures ranking granularity on the right (e.g., top- k , citation window). The central panel lists the top- k papers ranked by a milestone score, together with brief justifications, facilitating rapid triage for deeper study.

B Implementation Details of Knowledge Representation Layer

B.1 Knowledge Representation Layer Construction Pipeline

The knowledge graph construction pipeline forms the foundation of the Knowledge Representation Layer, transforming raw scholarly documents into a structured, semantically enriched graph that supports fine-grained retrieval and reasoning in AgenticScholar. The pipeline integrates *OCR-based text extraction*, *LLM-assisted segmentation and tagging*, *entity and relation extraction*, and *graph materialization*. The overall workflow proceeds through five main stages.

Stage 1: Document Parsing and Section-Level Chunking. For each research paper in PDF format, we first employ the *Mistral OCR API* [6] to rapidly convert the document into Markdown format for text analysis, and then apply *MinerU* [52] to detect visual content and extract high-resolution tables and figures with preserved layout. We adopt the *Mistral OCR API* and *MinerU* due to their strong efficiency in processing individual paper PDFs and their effectiveness in accurately

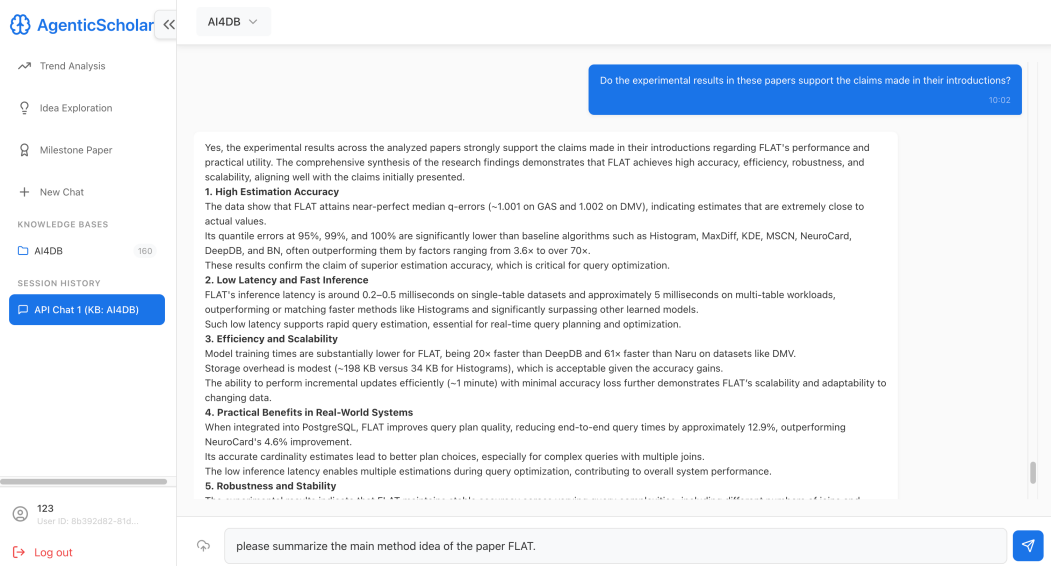


Fig. 10. Information Extraction and Synthesis.

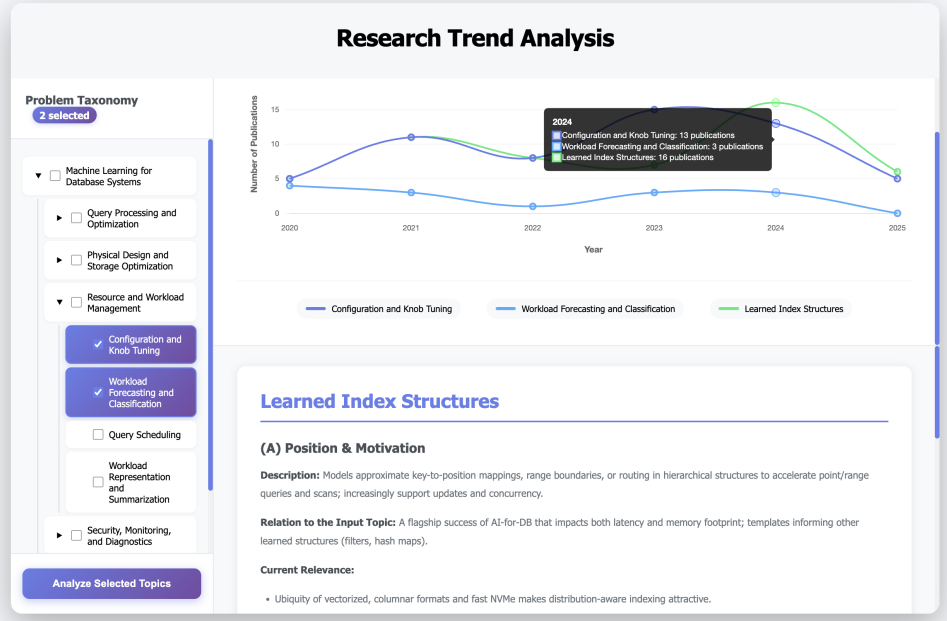


Fig. 11. Research trend analysis.

extracting tables and figures, as evidenced by the experimental results in Appendix G. To facilitate accurate downstream processing, we adopt a *section-level chunking strategy* that segments each paper by its natural sections and groups them into semantic units (e.g., Abstract, Introduction, Method, Experiment). We prioritize section boundaries because research papers are structurally self-contained and queries typically target specific parts (e.g., problem definition or experimental

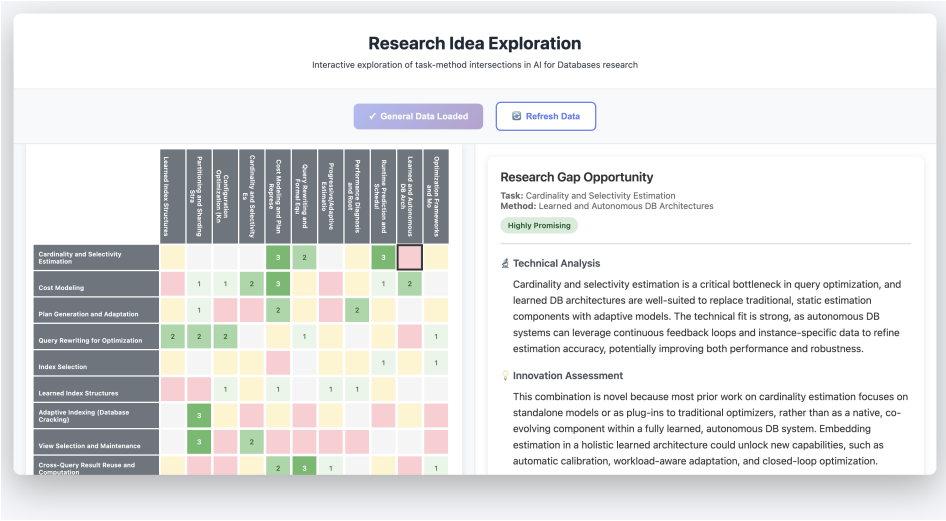


Fig. 12. Research idea exploration.

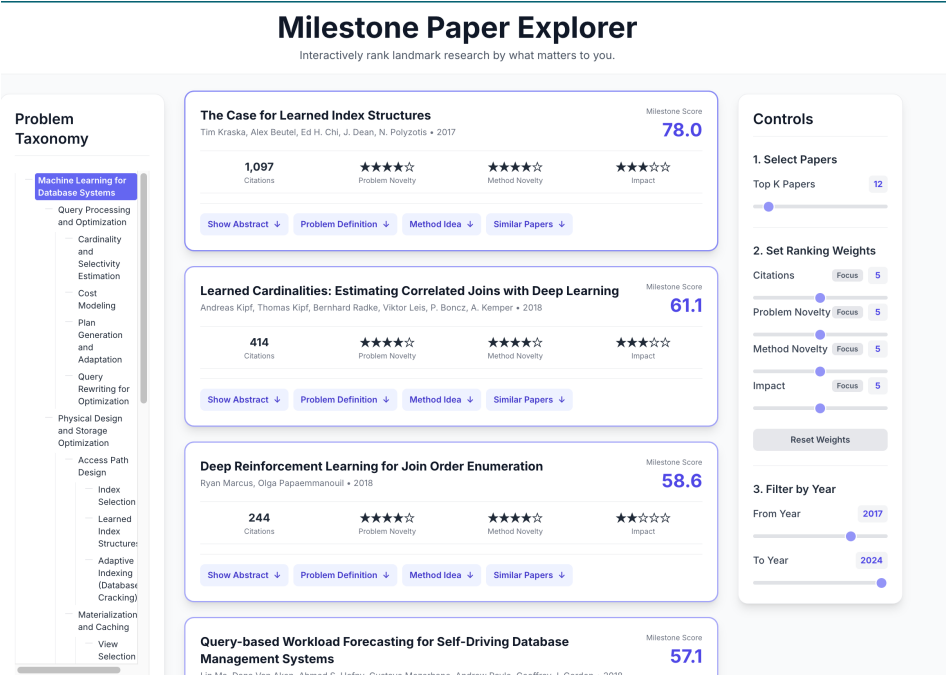


Fig. 13. Milestone paper selection.

setup), making this approach more effective than fixed windows or embedding-based topic segmentation [92] for scholarly analytics. To robustly detect sections across heterogeneous formats, we first match standardized patterns with regular expressions (numeric, Roman, alphabetic, or style-only) and extract the section titles. We then utilize an LLM as the classifier to infer the semantic label of each section title and maps it to standardized labels such as *Abstract*, *Introduction*, *Related*

Algorithm 3 BUILD TAXONOMY (End-to-End Pipeline)

Require: Corpus $\mathcal{D}$; taxonomy type κ (determines m aspects and extraction schema); optional reference taxonomy $\mathcal{T}_{\text{ref}}$ (or topic label l to generate it); match threshold τ_{match} ; refinement factor α ; max branches K_{max}

Ensure: Instantiated taxonomy $\mathcal{T}$; per-paper templates $S[\cdot]$; canonical aspect-class sets $\{\mathcal{A}_i\}_{i=1}^m$

```

1:  $m \leftarrow \text{NUMASPECTS}(\kappa)$ 
2:  $S \leftarrow \emptyset$ 
3: for all  $p \in \mathcal{D}$  do
4:    $S[p] \leftarrow \text{EXTRACTTEMPLATE}(p, \kappa)$ 
5:  $\{\mathcal{A}_i\}_{i=1}^m \leftarrow \text{CROSSPAPERSTANDARDIZATION}(\{S[p]\}_{p \in \mathcal{D}})$ 
6: if  $\mathcal{T}_{\text{ref}} = \text{null}$  then
7:    $l \leftarrow \text{INFERTOPICLABEL}(\mathcal{D})$ 
8:    $\mathcal{T}_{\text{ref}} \leftarrow \text{GENERATEREFTAXONOMY}(l, \kappa)$ 
9:  $\mathcal{T} \leftarrow \text{ALIGNANDINstantiate}(\mathcal{T}_{\text{ref}}, S, \{\mathcal{A}_i\}_{i=1}^m)$ 
10:  $\text{REGISTERUPDATEHANDLER}(\mathcal{T}, \kappa, \{\mathcal{A}_i\}_{i=1}^m, \alpha, \tau_{\text{match}}, K_{\text{max}})$ 
11: return  $(\mathcal{T}, S, \{\mathcal{A}_i\}_{i=1}^m)$ 

```

Work, Problem Formulation, Methodology, and Experiments. Sections sharing the same label are consolidated into the corresponding semantic unit. This strategy enables efficient, precise retrieval and aggregation for diverse queries.

Stage 2: Bibliographic Information Search. After text extraction, we retrieve bibliographic information for each paper using its title as a query to the *Semantic Scholar Paper Search API* [9]. The returned metadata—such as *authors* and *venue*—is incorporated into the knowledge graph as *bibliographic nodes*. This enrichment ensures consistency across the corpus and enables reasoning over scholarly provenance, co-authorship, and venue-specific trends.

Stage 3: Entity Extraction and Normalization. After obtaining the processed semantic units across sections, we feed them into LLM-based extractors to construct different categories of entities: (1) *Taxonomy nodes*: sections labeled as *Problem Formulation* and *Methodology* are used to extract formalized problem definitions and methodological strategies, forming the *Problem* and *Method* taxonomies described in Section 3.1; (2) *Experimental context nodes*: sections labeled as *Experiments* or *Evaluation* are used to extract datasets, metrics, and baselines, forming the corresponding Dataset, Metric, and Baseline nodes. Since different papers may refer to the same dataset or metric using variant names, we perform an LLM-based normalization step to unify semantically equivalent nodes across papers. This normalization ensures global consistency and enables cross-paper comparisons in higher-tier analytical queries.

Stage 4: Relation Construction and Schema Population. The extracted entities are linked to their corresponding Paper nodes through typed relations defined in the schema (Figure 4), such as ADDRESSES, APPLIES, USES, and HAS. These relations capture both intra-document and inter-document dependencies, situating each paper within a broader scholarly ecosystem.

Stage 5: Graph Materialization and Physical Storage. Finally, the complete structure is materialized as a *property graph* in Neo4j [19]. Each node and edge stores (i) symbolic attributes (e.g., names, descriptions, formal specifications) for rule-based filtering and graph traversal, and (ii) dense text embeddings for semantic similarity search. This hybrid representation supports both efficient traversal queries and embedding-driven retrieval. The Neo4j backend serves as the persistent substrate for higher-level components in AgenticScholar, including the hybrid query planner and the unified execution engine.

Algorithm 4 EXTRACTTEMPLATE (Structured Information Extraction)**Require:** Document p ; taxonomy type κ (defines aspect schema $\{(\text{name}_i, \text{instr}_i)\}_{i=1}^m$); LLM $\mathcal{M}$ **Ensure:** Structured template $S[p] = \langle \text{desc}_p, \text{aspect}_p^{(1)}, \dots, \text{aspect}_p^{(m)} \rangle$

```

1:  $m \leftarrow \text{NUMASPECTS}(\kappa)$ 
2:  $\mathcal{U} \leftarrow \text{SELECTSECTIONS}(p, \kappa)$ 
3:  $\text{ctx} \leftarrow \text{BUILDCONTEXT}(p, \mathcal{U})$  ▷ concatenate/trim to fit budget
4:  $\text{desc}_p \leftarrow \text{LLM\_SUMMARIZE}(\mathcal{M}, \text{ctx})$ 
5: for  $i = 1$  to  $m$  do
6:    $q_i \leftarrow \text{BUILDASPECTPROMPT}(\kappa, i)$ 
7:    $\text{aspect}_p^{(i)} \leftarrow \text{LLM\_EXTRACTASPECT}(\mathcal{M}, \text{ctx}, q_i)$ 
8:    $\text{aspect}_p^{(i)} \leftarrow \text{POSTPROCESS}(\text{aspect}_p^{(i)})$ 
9:  $S[p] \leftarrow \langle \text{desc}_p, \text{aspect}_p^{(1)}, \dots, \text{aspect}_p^{(m)} \rangle$ 
10: return  $S[p]$ 

```

Algorithm 5 CROSSPAPERSTANDARDIZATION**Require:** Set of per-paper templates $\{\langle \text{aspect}_p^{(1)}, \dots, \text{aspect}_p^{(m)} \rangle\}_{p \in \mathcal{D}}$ **Ensure:** Canonical aspect-class sets $\{\mathcal{A}_i\}_{i=1}^m$

```

1: for  $i = 1$  to  $m$  do
2:    $\mathcal{E}_i \leftarrow \emptyset$ 
3: for all  $p \in \mathcal{D}$  do
4:   for  $i = 1$  to  $m$  do
5:      $\mathcal{E}_i \leftarrow \mathcal{E}_i \cup \text{EXTRACTENTITIES}(\text{aspect}_p^{(i)})$ 
6: for  $i = 1$  to  $m$  do
7:    $\mathcal{V}_i \leftarrow \text{COMPUTEEMBEDDINGS}(\mathcal{E}_i)$ 
8:    $\mathcal{A}_i \leftarrow \text{CLUSTERANDLABEL}(\mathcal{V}_i)$ 
9: return  $\{\mathcal{A}_i\}_{i=1}^m$ 

```

B.2 Pseudo-code of Taxonomy Construction

This section presents the detailed pseudo code for our taxonomy construction algorithm, which forms the foundation for the hierarchical organization of scholarly knowledge in AgenticScholar. The pseudo code is presented in Algorithm 3.

The algorithm performs in four stages. (1) *Structured Information Extraction*: EXTRACTTEMPLATE extracts key aspects of the target concept into a structured template, leveraging relevant sections of the document. (2) *Cross-paper Standardization*: CROSSPAPERSTANDARDIZATION aligns the extracted aspects from templates to form unified canonical aspect-class sets $\{\mathcal{A}_i\}_{i=1}^m$ (where m depends on the taxonomy type). (3) *Reference Taxonomy Generation and Alignment*: A soft-guidance reference taxonomy $\mathcal{T}_{\text{ref}}$ is generated from the inferred topic label l and aligned with corpus-derived aspect signatures through ALIGNANDINSTANTIATE. (4) *Progressive Update and Branch Refinement*: UPDATETAXONOMY incrementally integrates newly arriving papers $\mathcal{D}_{\text{new}}$ under a size-controlled policy (α): each paper is routed to the most specific node and attached when similarity exceeds τ_{match} , otherwise a new branch is created; once a node becomes overpopulated ($\geq \alpha \cdot \text{BASESIZE}$), REFINEBRANCH applies corpus-grounded LLM_CLUSTERANDLABEL to split leaf nodes into finer subtopics or extend non-leaf nodes by adding a new sibling branch, and then reassigns papers accordingly. In the next, we will present the detailed algorithms for each stage.

Stage 1: Structured Information Extraction. This stage extracts key aspects from each paper and represents them in a structured template, including (i) a concise description and (ii) a taxonomy-dependent set of structured aspect fields $\{\text{aspect}^{(i)}\}_{i=1}^m$ that capture salient dimensions

Algorithm 6 ALIGNANDINSTANTIATE**Require:** Reference taxonomy $\mathcal{T}_{\text{ref}}$, signatures S , aspect classes $\{\mathcal{A}_i\}_{i=1}^m$ **Ensure:** Instantiated taxonomy $\mathcal{T}$

```

1:  $\mathcal{T} \leftarrow \text{COPYTREE}(\mathcal{T}_{\text{ref}})$ 
2:  $\mathcal{P} \leftarrow \text{DISTINCTTUPLES}(\text{MAPTOCLASSES}(S, \{\mathcal{A}_i\}_{i=1}^m))$ 
3: for all  $\mathbf{a} \in \mathcal{P}$  do  $\triangleright \mathbf{a} = (a_1, \dots, a_m)$ 
4:    $u \leftarrow \text{TOPDOWNLOCATEPARENT}(\mathcal{T}.\text{root}, \mathbf{a})$ 
5:    $v \leftarrow \text{BESTCHILDSIM}(u.\text{children}, \mathbf{a})$ 
6:   if  $\text{SIM}(v, \mathbf{a}) \geq \tau_{\text{match}}$  then
7:      $\text{target} \leftarrow v$ 
8:   else
9:      $(\text{name}, \text{desc}) \leftarrow \text{LLM\_NAMEANDDESCR}(\mathbf{a})$ 
10:     $\text{target} \leftarrow \text{ADDCHILD}(u, \text{Node}(\text{name}, \text{desc}, \mathbf{a}))$ 
11:   for all  $p \in \text{PAPERSWITH}(S, \mathbf{a})$  do
12:      $\text{target.papers} \leftarrow \text{target.papers} \cup \{p\}$ 
13: return  $\mathcal{T}$ 

```

Algorithm 7 UPDATETAXONOMY**Require:** Taxonomy $\mathcal{T}$, refinement factor α , match threshold τ_{match} **Ensure:** Updated taxonomy $\mathcal{T}$

```

1: on  $\text{NEWPAPER}(p)$ :
2:    $\text{sig} \leftarrow \text{EXTRACTSIGNATURE}(p)$ 
3:    $\mathbf{a} \leftarrow \text{MAPTOCLASSESORCREATE}(\text{sig})$   $\triangleright \mathbf{a} = (a_1, \dots, a_m)$ 
4:    $u \leftarrow \text{TOPDOWNLOCATEPARENT}(\mathcal{T}.\text{root}, \mathbf{a})$ 
5:    $v \leftarrow \text{FINDMATCHINGCHILD}(u, \mathbf{a}, \tau_{\text{match}})$ 
6:    $\text{target} \leftarrow v$  if  $v \neq \text{null}$  else  $\text{ADDCHILD}(u, \text{NodeFrom}(\mathbf{a}))$ 
7:    $\text{target.papers} \leftarrow \text{target.papers} \cup \{p\}$ 
8:   if  $\text{NEWPAPERCOUNT}(\text{target}) \geq \alpha \cdot \text{BASESIZE}(\text{target})$  then  $\text{REFINEBRANCH}(\text{target})$ 

```

for hierarchical organization. We leverage an LLM to analyze sections relevant to the target concept (e.g., abstract, introduction, and the most relevant definition/method sections). The procedure is detailed in Algorithm 4.

Stage 2: Cross-Paper Standardization. This stage unifies heterogeneous representations across papers by standardizing the extracted aspects. As shown in Algorithm 5, the procedure takes the collection of structured templates from Stage 1 and applies semantic canonicalization to each aspect dimension. Specifically, `EXTRACTENTITIES` identifies candidate terms from each aspect field; `COMPUTEEMBEDDINGS` encodes them into a shared vector space; and `CLUSTERANDLABEL` groups semantically similar terms to derive the canonical aspect-class sets $\{\mathcal{A}_i\}_{i=1}^m$. These sets ensure cross-paper consistency and enable downstream taxonomy alignment.

Stage 3: Reference Taxonomy Generation and Alignment. This stage integrates canonicalized aspect classes into a hierarchical structure guided by a draft reference taxonomy. As shown in Algorithm 6, the procedure first duplicates the reference tree via `COPYTREE` and maps each paper to a canonical aspect-signature tuple $\mathbf{a} = (a_1, \dots, a_m)$ using `MAPTOCLASSES`, where $a_i \in \mathcal{A}_i$. For each distinct tuple, the algorithm locates the most relevant parent node through `TOPDOWNLOCATEPARENT`, identifies the best-matching child using `BESTCHILDSIM`, and either reuses an existing node or invokes `LLM_NAMEANDDESCR` and `ADDCHILD` to instantiate a new one. Finally, all associated papers are linked to their corresponding taxonomy nodes.

Algorithm 8 REFINEBRANCH (Progressive Update: Branch Refinement)

Require: Taxonomy node u ; per-paper templates $S[\cdot]$; refinement threshold α ; match threshold τ_{match} ; max branches K_{max}

Ensure: Refined taxonomy structure around u

```

1: if NEWPAPERCOUNT( $u$ ) <  $\alpha$  then
2:   return
3:  $P \leftarrow \text{DIRECTPAPERS}(u)$ 
4: if  $|P| \leq 1$  then
5:   return
6:  $K \leftarrow \min(\text{ESTIMATESUBTOPICS}(P), K_{\text{max}})$ 
7: if  $K \leq 1$  then
8:   return
9:  $\{(C_1, \ell_1), \dots, (C_K, \ell_K)\} \leftarrow \text{LLM\_CLUSTERANDLABEL}(\{S[p]\}_{p \in P}, K)$ 
10: for  $k = 1$  to  $K$  do
11:   for  $i = 1$  to  $m$  do
12:      $\mathcal{A}_i^{(k)} \leftarrow \text{CANONICALIZE}(\{S[p].\text{aspect}^{(i)} \mid p \in C_k\})$ 
13:    $\mathbf{a}^* \leftarrow \text{DOMINANTTUPLE}(\{\mathcal{A}_i^{(k)}\}_{i=1}^m)$ 
14:   if ISLEAF( $u$ ) then
15:      $v \leftarrow \text{ADDCHILD}(u, \text{NodeFrom}(\ell_k, \mathbf{a}^*))$ 
16:   else
17:      $v \leftarrow \text{FINDMATCHINGCHILD}(u.\text{children}, \mathbf{a}^*, \tau_{\text{match}})$ 
18:     if  $v = \text{null}$  then
19:        $v \leftarrow \text{ADDCHILD}(u, \text{NodeFrom}(\ell_k, \mathbf{a}^*))$ 
20:    $\text{MOVEPAPERS}(C_k, v)$ 
21:  $\text{RESETNEWPAPERCOUNT}(u)$ 

```

Stage 4: Progressive Update and Refinement. This stage enables the taxonomy to evolve dynamically as new papers arrive. As shown in Algorithm 7, each incoming paper is converted into a structured signature by `EXTRACTSIGNATURE`, mapped to a canonical tuple $\mathbf{a}$ via `MAPTOCLASS-ESORCREATE`, and routed to the most specific node by `TOPDOWNLOCATEPARENT` and `FINDMATCHINGCHILD`. If no match exists, `ADDCHILD` instantiates a new node and the paper is attached. When a node accumulates enough new papers ($\geq \alpha \cdot \text{BASESIZE}$), we invoke `REFINEBRANCH` (Algorithm 8). Refinement partitions linked papers via `LLM_CLUSTERANDLABEL` and, for each cluster, derives a dominant canonical tuple $\mathbf{a}^*$ by aggregating evidence across all aspect dimensions. The taxonomy is then refined by adding or matching branches and reassigning papers accordingly, keeping the structure adaptive under concept drift.

C Predefined Plan Library

Table 12 presents our predefined plan library, which comprises 16 plans for executing the extract, summarize, rank, and check operators within individual scholarly documents, and 6 plans for applying the summarize, rank, and check operators across multiple documents.

D Details for Operators

In this section, we present the parameters and detailed implementation of each operator. All operators – except `Search`, `FindNode`, and `Traverse` – expose an `execution_mode` parameter (instance or group) that specifies whether the operator is applied to each item individually or to the entire collection as a whole. This design enables fine-grained control over execution behavior.

Table 12. Predefined plan library

Type	Operator	Plan description
Single doc.	Extract	1 Extract experimental settings
		2 Extract experimental datasets
		3 Extract evaluation metrics
		4 Extract compared baselines
		5 Extract the experimental results of comparison with baselines
		6 Extract the experimental results of parameter study
		7 Extract the experimental results of ablation study
		8 Extract the problem definition
		9 Extract the input of the problem
		10 Extract the output of the problem
		11 Extract the goal of the problem
		12 Extract the proposed method
Multiple doc.	Summarize	13 Summarize the impact of parameters based on experimental results
	Rank	14 Rank variants of the proposed method based on experimental results
		15 Rank experimental datasets based on experimental results
	Check	16 Check inconsistencies between the used evaluation metrics and the original evaluation metrics
	Summarize	17 Summarize common settings in experiments
		18 Summarize missing settings in experiments
		19 Summarize pros and cons across different methods
		20 Summarize differences among the problem definition
	Rank	21 Rank experiment results of different methods on a common dataset and a common evaluation metric
	Check	22 Check inconsistencies in experiment results on a common dataset and a common evaluation metric
Topic	Multiple operators	23 Research trend analysis
		24 Research idea exploration
		25 Milestone paper selection

D.1 Knowledge Access

Search. The Search operator performs ranked retrieval of scholarly documents from our knowledge graph.

Parameters. It takes a natural language query q as its primary input. The operator behavior is controlled by two main parameters: `candidates_per_query`, which specifies the number of initial candidates retrieved by each retrieval strategy to ensure high recall (default 20), and `top_k`, which defines the maximum number of the final ranked results returned to the user (default 10).

Implementation. The search operator is implemented as a five-stage pipeline that progressively refines the candidate set of papers to balance lexical precision and semantic relevance.

- i. *Query decomposition.* The input query is first parsed by an LLM-based component into structured attribute–value pairs, identifying which metadata fields and research aspects are relevant. We use five metadata fields = {*title*, *authors*, *affiliations*, *publication year*, *venue*}, and six research aspects = {*research topic*, *problem formulation*, *proposed method*, *experimental datasets*, *experimental baselines*, *experimental results*}.
- ii. *Temporal filtering.* Publication year constraints are applied as a strict preliminary filter to prune temporally invalid papers, reducing downstream computation.

- iii. *Hybrid candidate retrieval.* (1) Lexical matching on metadata: BM25 keyword search is applied to precise terms such as “authors” or “venue”; (2) Semantic search on research aspects: Cosine similarity search over pre-computed dense embeddings is used for descriptive research aspects such as “approximate nearest neighbor search” or “attribute/range filtering in high-dimensional vector spaces”.
- iv. *Score aggregation.* Scores from the two retrieval strategies are normalized and averaged within each group (BM25 for metadata, cosine similarity for research aspects), then combined with equal weight into a unified relevance score to prevent bias toward either lexical or semantic matches.
- v. *LLM-based reranking.* The top candidates from score aggregation are passed to an LLM-based reranker. This component jointly considers the original query together with the candidate’s metadata and research aspects, refining the final ranking and discarding results that are semantically plausible but contextually irrelevant.

D.2 Knowledge Navigation

FindNode. The FindNode operator is designed to locate specific *entry-point* nodes within the taxonomy nodes. The identified node will serve as the starting point for a more complex analytical plan.

Parameters. Its behavior is primarily controlled by one of two mutually exclusive parameters: `node_id`, which provides a specific identifier for direct and unambiguous retrieval, or `node_description`, a natural language phrase used to perform a semantic search to find the most relevant node in the taxonomy.

Implementation. If a `node_id` is provided, the system performs a direct, high-speed key lookup against the graph database. If a `node_description` is provided instead, the system performs a semantic similarity search over the embeddings of the taxonomy nodes’ descriptions to identify and return the best match.

Traverse. The Traverse operator is the primary navigation function of the knowledge graph. It enables movement from a given set of starting nodes to new sets of connected nodes by following specified relationships.

Parameters. The operator takes a collection of `start_nodes` (produced by a preceding operator) as input. Its behavior is governed by the `traversal_path` parameter – a structured list that defines the sequence of relationship types to follow and the target entity types to retrieve at each step along the graph.

Implementation. It takes the `traversal_path` and translates it into a single, efficient, multi-hop query in the native language of the underlying knowledge graph engine (e.g., a Cypher query with multiple MATCH clauses). This query is then executed against the database, and the results are formatted into a standard list of entities for consumption by subsequent operators.

Retrieve. This operator fetches the raw content of specific sections from a document.

Parameters. The operator is controlled by a `document_id` and `section_tags` (e.g., “Introduction”, “Methodology”) to precisely locate and return the required text for downstream processing.

D.3 Semantic Content Processing

Extract. This operator takes raw text content (typically from a Retrieve operator) and extracts specific, structured information from it. It is controlled by an instruction parameter, which provides details to guide LLM’s extraction logic, and `detail_level`, including short, detailed, or detailed with evidence. Unlike a simple retrieval function that returns raw text, the Extract operator interprets a user’s query to identify and return precise details, such as experimental settings.

Parameters. The operator is invoked with a natural language query, q , and requires identifiers for the target document, `document_id`, and the specific section_tags from which to extract information. Its behavior is controlled by an optional `extract_instruction` parameter, which provides a detailed template to guide the LLM's extraction logic and output format. AgenticScholar offers a library of predefined plans (See Table 12) for common scholarly queries, such as extracting experimental settings or problem definitions, while also allowing users to supply custom instructions for their own extraction needs.

Implementation. The operator's execution follows a three-stage pipeline:

- i. *Section retrieval*: The operator first retrieves the full text content of the section specified by `section_tags` from the document identified by `document_id`.
- ii. *Prompt construction*: A dynamic prompt is then constructed by integrating the retrieved section content, the user query q , and the `extract_instruction`. The user query q provides high-level intent, while the `extract_instruction` provides a low-level template that directs the LLM on precisely what to extract and how to structure it (e.g., as a JSON object with predefined keys).
- iii. *LLM generation*: The composed prompt is dispatched to an LLM. The model processes the input and generates a structured response containing the specific information requested by the user.

Summarize. This operator is designed to condense textual information from one or more inputs into a coherent and concise overview. Its behavior is controlled by an instruction parameter, which guides the LLM's summarization logic. Its primary purpose is to reduce lengthy content, optimize context for subsequent processing, and distill key insights/aspects tailored to a user's specific focus.

Parameters. The operator is invoked with the original user query, q , and requires a collection of `input_data` from a preceding operator(s) as its primary input. Its summarization behavior is controlled by two main parameters: `detail_level` and `focus`. The `detail_level` parameter dictates the output's length and depth, with options for a short overview (`short`), a detailed summary (`detailed`), or a summary that includes supporting evidence from the source texts (`detailed_with_evidence`). The `focus` parameter provides a specific thematic instruction that guides the summary. Same as Extract, AgenticScholar offers a library of predefined plans (See Table 12) for common scholarly queries, such as focusing on the "pros and cons across different methods", while also allowing users to supply custom instructions to meet their own summarization needs.

Implementation. The implementation is in a straightforward manner: the system constructs an appropriately parameterized prompt over the operator's input data and issues a single LLM call to obtain the result. In the remainder of this paper, we omit explicit implementation details for operators that are realized in this same manner.

Check. This analytical operator compares information across multiple peer documents to identify and report on inconsistencies or disagreements, guided by instruction parameter.

Parameters. The operator is invoked with the original user query, q , and requires a list of two or more contents from preceding operator(s) as its primary input to serve as the peer sources for comparison. Its behavior is controlled by the `check_instruction` parameter, which defines the instructions to guide the checking process.

Verify. This operator fact-checks a specific claim against a designated source of evidence, operating either within a single paper for internal consistency or across papers under the guidance of the instruction parameter.

The Verify operator is an analytical function designed to fact-check a specific claim against a designated source of evidence. It is a versatile tool that can operate both within a single paper to check for internal consistency and across multiple papers to validate citations and reported findings.

Parameters. The operator is invoked with the original user query, q , and requires two direct string inputs: a claim and the evidence generated from preceding operator(s).

Rank. This operator is a specialized function designed to evaluate and order a set of entities from the `input_data` parameter based on a specific, measurable criterion defined in the instruction parameter, producing a structured, ordered list as its primary output.

The Rank operator is a specialized function designed to evaluate and order a set of entities based on a specific, measurable criterion, producing a structured, ordered list as its primary output. It can be used within a single paper, such as ranking variants of a proposed method based on ablation study results, or it can be applied across multiple papers, such as ranking different approaches based on their reported performance on a common benchmark.

Parameters. The operator is invoked with the original user query q and a collection of `input_data` produced by preceding operators. Its ranking behavior is governed by the parameter `rank_instruction`, which may either (i) directly specify the entities to be ranked and the ranking criterion, or (ii) provide instructions on how these entities or criteria should be identified from the input. In addition, users may supply auxiliary aspects to annotate the ranking results, enabling richer comparative outputs, and `ranking_criterion`. The `entities_to_rank` parameter specifies the set of items to be ordered, such as “variants of the proposed method” or “datasets used in the same method”. The `ranking_criterion` parameter defines the specific metric for the comparison, for instance, “experimental results on a common dataset and a common evaluation metric”.

D.4 Relational-Style Data Manipulation

GroupBy. This operator partitions a list of entities in `input_data` based on a specified attribute. Its `grouping_key` specifies the attribute or attributes to be used for partitioning the data.

The GroupBy operator takes a list of entities and restructures it into a nested, grouped format based on specified attributes.

Parameters. The operator takes a collection of `input_data` from a preceding operator as its primary input. Its behavior is controlled by the `grouping_key` parameter, which specifies the attribute or attributes to be used for partitioning the data.

Aggregate. This operator computes summary statistics over a set of entities from `input_data`. The instruction specifies the function (e.g., COUNT, MAX) and the target of the function (e.g., a node’s attributes), which may require semantic interpretation.

Parameters. The operator takes a collection of `input_data` from a preceding operator as its primary input. Its behavior is controlled by the `aggregation_instruction` parameter, which is a structured object that specifies the exact calculation to be performed. This instruction defines the function (e.g., COUNT, MAX, AVG), the target of the function (e.g., a node’s properties or its relationships), and any required other specifics.

Filter. This operator returns a subset of entities from a collection that satisfies a logical condition. The condition is provided via a `filter_instruction` parameter, which is evaluated semantically by an LLM for each entity in the `input_data`.

The Filter operator is a selection function to take a list of entities and return a subset that satisfies a specific, logical criterion.

Parameters. The operator takes a collection of `input_data` from a preceding operator as its primary input. Its behavior is controlled by the `filter_instruction` parameter, which describes the logical condition to be applied.

D.5 Knowledge Generation and Synthesis

Generate. The Generate operator is a versatile, LLM-powered function designed to produce new, coherent text based on the `input_data` and a specific instruction.

Parameters. The operator takes a collection of `input_data` from preceding operators as its primary contextual input. Its behavior is controlled by the `generation_instruction` parameter, which is a detailed, task-specific prompt or template.

MatrixConstruct. A specialized operator designed for research idea exploration. It synthesizes information on problems and methods from a research area into a structured matrix, enabling the systematic discovery of under-explored combinations.

Parameters. The operator takes a collection of `input_data` from preceding operators as its primary input. It automatically parses this input to identify the problem and method taxonomies, mapping problems to matrix rows and methods to matrix columns. This organization forms the grid structure required to identify potential research gaps.

E Implementation of Query Processing Pipelines

This section details the implementation of query processing pipelines for complex analytical queries (Tier 3), which are constructed using the proposed operators.

E.1 Research Trend Analysis

The research trend analysis query processing pipeline integrates taxonomy navigation, external retrieval, statistical aggregation, and LLM-based synthesis to construct a comprehensive view of emerging research dynamics. Specifically, with the Traverse operator, each leaf node in the problem or method taxonomy is first identified to capture its contextual hierarchy within the knowledge graph. The Search operator then issues structured API queries to external scholarly sources, Semantic Scholar, retrieving publications that match the node name or its semantically expanded variants. Retrieved metadata, including publication year and citation count, is subsequently processed by the Aggregate operator to compute temporal distributions and citation growth metrics across recent years.

Unlike traditional numeric ranking, AgenticScholar employs an LLM-centric strategy for trend evaluation. The aggregated statistics are passed directly to the Generate operator, where an LLM interprets the multi-dimensional trend data – combining publication growth, citation velocity, and topical coherence – to both *rank* subtopics and *summarize* their significance. This unified semantic reasoning enables a more nuanced prioritization of emerging subfields, highlighting the top-*k* trends within each topic together with concise, evidence-grounded explanations of what drives their recent prominence.

E.2 Research Idea Exploration

The research idea exploration query processing pipeline supports systematic discovery of under-explored research directions by reasoning over the cross-product space of problem and method taxonomies. The MatrixConstruct operator first builds a 2D matrix where each cell corresponds to a possible (method, problem) pairing. The Filter operator examines the knowledge graph to detect existing publications aligned with these pairs and removes those already explored, yielding a candidate set of unexplored intersections.

To evaluate and prioritize these intersections, AgenticScholar introduces a *two-stage LLM-driven assessment process*. In the first stage, the LLM performs a lightweight scoring over all unexplored combinations, guided by compact descriptors of the method's capability, e.g., strengths and weaknesses, and the problem's characteristics, e.g., the input and output. The LLM estimates the

Table 13. Topics used in our evaluation.

Topic Type	Topics
Broad Domains	AI for Databases
Specialized Sub-fields	Query Optimization; Database Design & Execution; Cross-Modal Retrieval; Dataset Discovery; Entity Resolution; Health IR; Learned Indexes; Recommendation; Time-Series Forecasting; Time-Series Similarity Search; Trajectory Search
Emerging Areas	Vector Search; LLMs for Structured Data Analysis

plausibility and potential impact of applying the given method to the task, outputting a normalized relevance score that enables efficient ranking and selection of the top- k promising combinations. In the second stage, each selected pair undergoes a deeper generative reasoning phase: the LLM further produces a structured, insight-rich research proposal outlining motivation, feasibility, novelty, and potential contributions.

This hierarchical design balances scalability and analytical depth: the first stage provides broad exploration across a large combinatorial space, while the second stage offers focused, high-fidelity synthesis. Collectively, this two-phase pipeline transforms sparse knowledge signals into actionable research hypotheses, supporting a principled and data-driven workflow for identifying novel problem–method intersections within AgenticScholar.

E.3 Milestone Paper Selection

The milestone paper identification processing pipeline focuses on recognizing highly influential works that have significantly shaped a research field. It integrates bibliometric data, taxonomic novelty signals, and temporal impact dynamics to assign a composite milestone score to each candidate paper. Specifically, AgenticScholar retrieves papers within the target topic through Search and aggregates relevant metadata—such as publication year, citation history, and taxonomy alignment—via the Aggregate operator.

Each paper is evaluated across four primary dimensions: (i) *citations*, reflecting normalized citation volume and trajectory; (ii) *problem novelty*, measuring how early and distinctively the paper introduced a new problem formulation in the taxonomy; (iii) *method novelty*, capturing the originality of algorithmic techniques and their contextual innovation; and (iv) *impact*, quantifying both early influence and sustained citation momentum. Additionally, a delayed recognition adjustment identifies “sleeping beauty” papers that gained influence belatedly and applies an appropriate novelty boost. After normalization, the results are ranked via the Rank operator, and the top- k milestone papers are synthesized by the Generate operator into concise summaries outlining their novelty, influence, and long-term significance.

F Selected Topics for Experiments

Our experimental design adopts a stratified topic selection to ensure a comprehensive evaluation of AgenticScholar’s versatility across diverse scholarly domains. As shown in Table 13, we categorize topics into three types: (1) *broad domains*, representing overarching areas that encompass multiple sub-fields; (2) *specialized sub-fields*, focusing on specific technical problems within a domain; and (3) *emerging areas*, capturing rapidly evolving research frontiers. This stratification enables coverage from large, foundational domains to focused, mature sub-fields. Importantly, including emerging areas allows us to assess AgenticScholar’s ability to reason over novel, corpus-specific information that may not yet be well-represented in an LLM’s internal knowledge.

We tailor the number of topics to each evaluation’s objective. For foundational retrieval (Tier 1), we use all 10 specialized sub-field topics (see Table 14 for per-topic results) to ensure robust

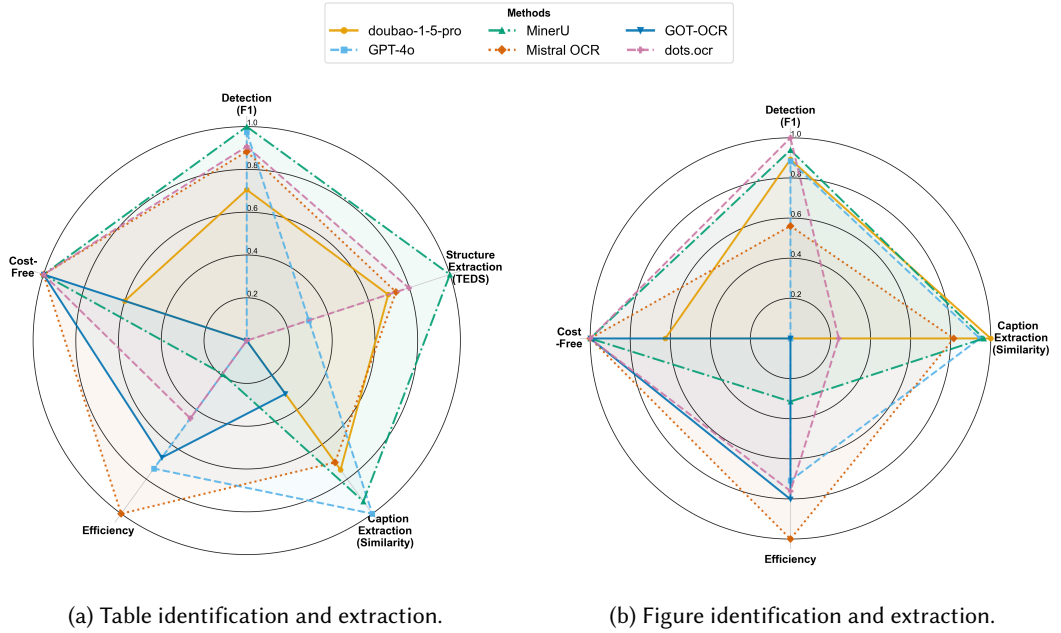


Fig. 14. Comparison of document parsing tools across effectiveness, efficiency, and monetary cost. “Cost-Free” denotes the normalized monetary-cost axis (higher is better, with free tools achieving the maximum score). benchmarking. For information extraction and synthesis (Tier 2) and knowledge discovery and generation (Tier 3), we use a smaller but more diverse set of five topics spanning all three categories. This setup rigorously tests the system’s generality and reasoning ability across the full spectrum of scholarly domains. Specifically, the *broad domain* includes AI for Databases; the *specialized sub-fields* comprise Query Optimization and Database Design and Execution; and the *emerging areas* cover Vector Search and LLMs for Structured Data Analysis, reflecting the growing intersection between database systems and modern AI techniques.

G Evaluation of Document Parsing Tools

In this section, we evaluate different document parsing tools based on their *effectiveness* in correctly identifying and extracting tables/figures and their *efficiency* and *monetary cost* in processing individual paper PDFs.

G.1 Experimental Setup

Dataset. We use 94 research papers from the *academic_literature* split of OmniDocBench [61] that contain at least one table or figure. For each paper, OmniDocBench provides a sampled page with tables/figures and fine-grained page-level ground-truth annotations, including: (i) table bounding boxes, table HTML, and table attributes (e.g., layout type, cell spans, formulas); (ii) figure bounding boxes; and (iii) captions and footnotes for both tables and figures. We treat these annotations as ground truth.

Compared Methods. We compare representative tools from three categories: (i) a pipeline-style document analysis tool, MinerU [52]; (ii) specialized OCR tools, Mistral OCR [6], GOT-OCR [86], and dots.ocr [44]; and (iii) general-purpose VLMs, doubao-1.5-thinking-vision-pro (doubao-1-5-pro) [29] and GPT-4o [33].

Implementation. Pipeline/OCR tools take the raw PDF page as input, whereas VLMs take the rendered page image from the PDF page. All outputs are normalized into a structured format that contains, for each detected element, (i) the table/figure region (bounding box), (ii) table HTML capturing structure and cell content, and (iii) captions and footnotes.

Evaluation Metrics. Following OmniDocBench [61], we assess *effectiveness* for table/figure extraction via both detection and extraction quality: (i) *Detection*: we compute table/figure F1 by matching predictions to ground-truth instances with $\text{IoU} \geq 0.5$; (ii) *Structure and Caption Extraction*: for matched elements, we evaluate table structure extraction quality using TEDS (Tree-Edit-Distance-based Similarity) between predicted and ground-truth HTML, and caption extraction quality for both tables and figures using embedding-based cosine similarity between predicted and ground-truth captions. For *efficiency*, we report the average runtime per paper page. For API-based methods, we additionally report the average *monetary cost* per paper page based on logged token usage and provider pricing.

G.2 Evaluation Results

Figure 14 compares document parsing tools in terms of effectiveness, efficiency, and monetary cost, based on a normalized radar chart (higher is better on all axes). Specifically, for effectiveness metrics, we use standard min-max scaling, $\text{Score}(x) = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$; for efficiency and monetary cost, we use the inverted form, $\text{Score}(x) = \frac{x_{\max} - x}{x_{\max} - x_{\min}}$, so lower latency and lower cost yield higher scores. From the results, we can observe that MinerU achieves the strongest overall effectiveness for identifying and extracting tables and figures, but at a noticeably higher runtime, whereas Mistral OCR offers the best efficiency-effectiveness trade-off, delivering competitive extraction quality with substantially lower latency. In contrast, general-purpose VLMs (e.g., GPT-4o and doubao-1.5-pro) incur non-trivial API costs without providing proportional effectiveness gains. Based on these observations, we adopt a hybrid preprocessing strategy: we use the *Mistral OCR API* for rapid, low-latency conversion and deploy *MinerU* as the default backend when high-fidelity, open-source table/figure extraction is required.

H Supplements for Evaluation of Retrieval

H.1 Supplements for Query Generation

Query Types. We consider two categories of queries for evaluation:

- *Factual queries* target explicit facts concerning query attributes, such as paper metadata or specific research aspects, including: (i) *single-attribute* queries (e.g., “which papers use the UCR dataset?”) and (ii) *multi-attribute* queries (e.g., “which SIGMOD papers propose transformer-based models for time series forecasting after 2020?”).
- *Analytical queries* involve comparative or reasoning-based understanding including: (i) *in-topic comparative* queries (e.g., “which papers achieve higher accuracy than DTW on time series forecasting?”) and (ii) *cross-topic comparative* queries spanning multiple topics (e.g., “which papers evaluate transformer-based methods on tasks beyond natural language processing?”).

Query and Ground Truth Generation Prompts. We design two structured prompt templates (Figures 15 and 16) to generate natural language queries and identify corresponding ground truth sets from the paper corpus. Each prompt is conditioned on summaries of paper attributes, including metadata (e.g., title, authors, publication year, etc.) and research aspects (e.g., research topic, proposed method, experimental datasets, etc.). The first template constructs factual queries and analytical queries confined to a single research topic, while the second focuses on analytical queries that generalize across topics. Both templates instruct the LLM to output (query, ground truth) pairs, which are subsequently verified for correctness and completeness.

Table 14. Per-topic effectiveness across models.

Topic	BM25		GTR		E5		Instructor		Emb.Gm		ColBERT		Qwen3		Ours	
	R-P	MAP	R-P	MAP	R-P	MAP	R-P	MAP	R-P	MAP	R-P	MAP	R-P	MAP	R-P	MAP
Cross-Modal Ret.	0.50	0.56	0.51	0.54	0.47	0.55	0.51	0.51	0.45	0.47	0.60	0.61	<u>0.67</u>	<u>0.69</u>	0.80	0.81
Dataset Discovery	0.54	0.59	0.43	0.43	0.49	0.49	0.47	0.48	0.52	0.55	0.55	0.57	<u>0.56</u>	<u>0.63</u>	0.86	0.88
Entity Resolution	0.45	0.47	0.36	0.35	0.39	0.40	0.42	0.46	0.40	0.41	0.44	0.45	<u>0.48</u>	<u>0.49</u>	0.60	0.62
Health IR	0.68	0.71	0.45	0.50	0.62	0.67	0.50	0.54	0.60	0.63	0.69	0.71	<u>0.74</u>	<u>0.77</u>	0.85	0.90
Learned Index	0.50	0.48	0.28	0.31	0.54	0.52	0.50	0.44	0.42	0.41	<u>0.62</u>	0.59	0.58	<u>0.60</u>	0.71	0.72
Recommendation	0.35	0.43	0.45	0.48	0.57	0.57	0.51	0.52	0.46	0.47	0.56	0.56	<u>0.62</u>	<u>0.64</u>	0.66	0.70
TS Forecasting	0.45	0.43	0.48	0.53	0.49	0.51	0.52	0.54	0.45	0.44	0.57	<u>0.58</u>	<u>0.58</u>	<u>0.58</u>	0.74	0.78
TS Sim. Search	0.41	0.44	0.23	0.27	0.30	0.38	0.35	0.40	0.39	0.41	0.43	0.45	<u>0.48</u>	<u>0.51</u>	0.55	0.60
Trajectory Search	0.37	0.52	0.35	0.44	0.42	0.53	0.57	0.62	0.50	0.53	0.61	0.65	<u>0.61</u>	<u>0.69</u>	0.79	0.77
Vector Search	0.37	0.39	0.30	0.34	0.39	0.40	0.30	0.33	0.36	0.35	0.49	0.49	<u>0.50</u>	<u>0.51</u>	0.68	0.64
Cross-Topic Comp.	0.46	0.35	0.45	0.39	<u>0.52</u>	<u>0.46</u>	<u>0.52</u>	0.43	0.41	0.38	0.38	0.35	0.44	0.44	0.56	0.51
Mean	0.46	0.50	0.39	0.42	0.47	0.50	0.47	0.48	0.45	0.46	0.53	0.53	<u>0.57</u>	<u>0.59</u>	0.71	0.72

Table 15. Survey papers used for benchmark construction

Paper Title	First Author et al.	Year	Domain
Instruction Tuning for Large Language Models: A Survey	Zhang et al.	2025	ML
A Primer on Temporal Graph Learning	Rahman et al.	2025	
Multi-Step Reasoning with Large Language Models, a Survey	Plaat et al.	2025	
Indexing Techniques for Graph Reachability Queries	Zhang et al.	2025	DB
A Survey of Learned Indexes for the Multi-dimensional Space	Al-Mamun et al.	2025	
Review of Explainable Graph-Based Recommender Systems	Markchom et al.	2025	IR
Machine Learning Systems: A Survey from a Data-Oriented Perspective	Cabrera et al.	2025	SE

H.2 Evaluation Results

Table 14 reports per-topic R-Precision and MAP across all retrieval models. AgenticScholar consistently outperforms both lexical and dense baselines across all domains. The improvement is consistent even in analytically demanding queries (i.e., *Cross-Topic Comparative*), demonstrating that AgenticScholar generalizes effectively across heterogeneous research topics.

I Supplements for Evaluation of Information Extraction and Synthesis

Figure 17 shows additional results of Tier-2 queries.

J Supplements for Evaluation of Knowledge Discovery and Generation

Query Generation. For each topic, we design three concise prompt types to form the query, covering distinct Tier-3 objectives: research trend analysis, research idea exploration, and milestone paper selection, as presented in Figures 18, 19, 20, and 21.

- **Milestone Paper Selection:** Identifies landmark works since a given year, emphasizing problem or method novelty and long-term impact through influence or citations.
- **Research Trend Analysis:** Analyzes the most recent 2–3 years of literature to reveal emerging *problems* and *methods* within a topic, explaining their motivation, significance, and driving factors such as new models, applications, or environments.
- **Research Idea Exploration:** Encourages creative brainstorming of new research questions, summarizing each idea’s core concept, novelty, and key technical challenge.

Table 16. Evaluation on taxonomies (Ass.: Assignment Accuracy; Hie.: Hierarchical Consistency; Sib.: Sibling Coherence).

Level	Methods	Problem taxonomy			Method taxonomy		
		Ass.	Hie.	Sib.	Ass.	Hie.	Sib.
1	TaxoAdapt	0.76	0.41	0.25	0.87	0.61	0.25
	AgenticScholar _{noC}	0.76	0.96	1.00	0.74	0.76	0.00
	AgenticScholar _{noR}	0.90	0.77	0.75	0.93	0.75	0.50
	AgenticScholar	0.84	0.97	1.00	0.90	0.87	1.00
2	TaxoAdapt	0.85	0.72	0.29	0.95	0.80	0.79
	AgenticScholar _{noC}	0.66	0.98	1.00	0.79	0.76	0.76
	AgenticScholar _{noR}	0.84	0.94	1.00	0.92	0.76	0.96
	AgenticScholar	0.81	0.97	1.00	0.87	0.82	0.97
3	TaxoAdapt	0.95	0.67	0.67	0.86	0.63	0.60
	AgenticScholar _{noC}	0.47	0.85	0.90	0.78	0.80	0.91
	AgenticScholar _{noR} [†]	–	–	–	–	–	–
	AgenticScholar	0.80	0.92	1.00	0.94	0.77	0.92

[†] Since AgenticScholar_{noR} typically yields shallow structures and no level-3 nodes, its level-3 results are omitted.

K Supplements for Validating Architectural Design

K.1 Quality Evaluation of Taxonomies

K.1.1 Benchmark Construction For Quantitative Analysis. Following [96], we construct the ground-truth taxonomy by manually extracting expert-curated hierarchies from authoritative survey papers for each topic and validating the resulting nodes and parent–child relations. We select high-quality surveys from top venues (e.g., CSUR) spanning *Databases (DB)*, *Machine Learning (ML)*, *Software Engineering (SE)*, and *Information Retrieval (IR)*. We instantiate the taxonomy over the paper corpus covered by these surveys. The survey sources are listed in Table 15.

K.1.2 LLM-judged Quality Evaluation. As discussed in Section 3, the knowledge representation layer organizes the papers into hierarchical taxonomies.

Baselines. We evaluate their quality against (1) *TaxoAdapt* [36], which is a state-of-the-art method that refines LLM-generated taxonomies for corpus alignment. (2) two ablated variants of our reference-enhanced taxonomy construction method, AgenticScholar_{noC} and AgenticScholar_{noR}, which differ in taxonomy construction: AgenticScholar_{noC} directly adopts the LLM-generated reference taxonomies, whereas AgenticScholar_{noR} derives them entirely from the corpus — i.e., solely from the input papers without any reference guidance.

Evaluation Metrics. For each taxonomy, we use GPT-5 as the LLM evaluator (prompts in Figures 24 and 25) and assess its quality across the four metrics following [36]:

- **Assignment Accuracy.** This metric measures whether each node’s assigned papers are predominantly relevant. For each node n_i with an assigned paper set S_i , let $y_{ij} \in \{0, 1\}$ denote the evaluator’s binary judgment that paper $j \in S_i$ is semantically relevant to n_i . A node is counted as correct if a strict majority (i.e., $> 50\%$) of its assigned papers are labeled relevant. The metric is calculated as $\text{Acc}_{\text{assign}} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}\left[\frac{1}{|S_i|} \sum_{j \in S_i} y_{ij} > 0.5\right]$, where $\mathbb{I}[\cdot]$ is the indicator function.
- **Hierarchical Consistency.** This metric evaluates whether each child node is semantically subsumed by its parent. Let $\mathcal{PC}$ denote the set of directed parent–child pairs (n_p, n_c) , the metric is calculated as $\text{Cons}_{\text{hier}} = \frac{1}{|\mathcal{PC}|} \sum_{(n_p, n_c) \in \mathcal{PC}} \text{LLMScore}(n_p, n_c)$, where $\text{LLMScore}(n_p, n_c)$ is the evaluator’s binary judgment that the concept of n_p meaningfully generalizes that of n_c .

- **Sibling Coherence.** This metric evaluates the conceptual coherence of sibling groups (children under the same parent). Let P be the set of parent nodes, and for each $p \in P$, let $C_p = \{n_1, \dots, n_{k_p}\}$ denote its sibling group. This metric is calculated as $\text{Coh}_{\text{sib}} = \frac{1}{|P|} \sum_{p \in P} \text{LLMScore}_{\text{sib}}(C_p, p)$, where $\text{LLMScore}_{\text{sib}}(C_p)$ is the evaluator's binary judgement that the nodes in C_p are semantically comparable and instantiate the same organizing aspect relative to p .
- **Description Accuracy.** This metric measures how accurately each node's textual description captures its intended concept. For a set of taxonomy tree nodes $\{n_i\}_{i=1}^N$, the metric is defined as $\text{Acc}_{\text{desc}} = \frac{1}{N} \sum_{i=1}^N \text{LLMScore}(d_i, c_i)$, where $\text{LLMScore}(d_i, c_i) \in \{0, 1\}$ indicates whether the evaluator judges the extracted attributes d_i of node n_i to be semantically correct with respect to the canonical concept c_i .

Results. Given the hierarchical nature of the taxonomy, we report quality metrics by depth and average them across constructed trees. Detailed Description Accuracy is omitted since all methods are near-perfect (≈ 1). The results are shown in Table 16. Observe that: (1) AgenticScholar consistently surpasses TaxoAdapt on Hierarchical Consistency and Sibling Coherence, indicating stronger structure from cross-paper standardization and reference alignment. Note that AgenticScholar shows slightly lower Assignment Accuracy in a few cases (e.g., problem-level 3). This stems from TaxoAdapt's fragmentation (e.g., many nodes with 2 papers), which can inflate the accuracy, whereas AgenticScholar consolidates papers into fewer nodes for better structural quality. (2) AgenticScholar maintains uniformly strong results compared to its two variants, confirming that fusing corpus-grounded evidence with an LLM scaffold yields higher-quality taxonomies.

Case Study. To further qualitatively assess taxonomy quality, we conduct a case study using the problem taxonomy of the topic "Vector Search". As illustrated in Figure 23, AgenticScholar produces a more coherent, semantically organized hierarchy with more consistent sibling groupings, and closer alignment to source papers than other approaches.

K.2 Case Study for Self-correction

Figure 22 presents a real case from our testing process. For the researcher's query: "From the set of papers that study graph-based methods on the hybrid vector search problem, identify the two best-performing methods on the SIFT1M dataset in terms of the QPS metric. What specific algorithmic differences between these two methods most likely explain the performance gap?", the system initially generated a plan tree as shown in the lower-left corner. Plan Validator then inspected the plan and identified the following major issues:

- **Step-internal issues:** issues in internal consistency within each step.
 - "The handler_type 'extract_exp_settings' suggests extracting experimental settings, but the description focuses on extracting algorithmic details and architectures from 'Method' sections. This is inconsistent."
 - "The handler_type 'rank_datasets' does not clearly correspond to ranking methods by QPS performance."
- **Inter-step issues:** issues in logical coherence between steps.
 - "step_3 depends on step_1 (retrieval of 'Experiment' sections) but aims to extract from 'Method' sections. Since step_1 only retrieves 'Experiment' sections, step_3 lacks the required input."
- **Overall plan issues:** issues in the completeness or correctness of the entire plan.
 - "The final step is a 'summarize' step that produces a summary, which is appropriate, but the plan lacks an explicit step to format or present the final answer as a concise comparison or ranked list as implied by the query."

Table 17. The impact of backbone LLMs for research trend analysis. The colors express relative values (*worse than average*, *average*, and *better than average*).

Metric	Claude 4.5 Sonnet	Claude 4.5 Opus	GPT-5 Mini	GPT-5	Gemini-3.0 Flash	Gemini-3.0 Pro	Grok-4.1 Fast (NR)	Grok-4.1 Fast (R)	Avg.	Std. Dev
<i>Problem-oriented</i>										
Token cost	11,270	11,645	10,328	11,345	9,915	11,203	9,908	13,650	11,158	±10.9%
Monetary cost (\$)	0.164	0.282	0.021	0.110	0.030	0.130	0.005	0.007	0.094	±104.3%
Efficiency (s)	402.6	531.2	255.8	387.9	243.9	416.7	107.6	190.3	317.0	±44.2%
Correctness	7.74	7.9	7.48	8.08	7.56	7.98	7.18	7.28	7.65	±4.3%
Relevance	8.7	8.84	8.32	9.08	8.42	8.92	7.96	8.06	8.54	±4.8%
Diversity	7.62	7.78	7.42	7.92	7.54	8.02	7.12	7.22	7.58	±4.2%
Specificity	8.22	8.36	7.88	8.62	7.98	8.48	7.52	7.62	8.09	±5.0%
<i>Method-oriented</i>										
Token cost	17,397	18,150	16,552	17,463	15,760	17,652	15,134	21,605	17,464	±11.2%
Monetary cost (\$)	0.052	0.090	0.007	0.035	0.009	0.042	0.001	0.002	0.030	±105.1%
Efficiency (s)	357.4	518.5	231.3	370.9	221.4	391.8	133.6	216.9	305.2	±41.0%
Correctness	7.66	7.82	7.38	7.96	7.44	7.88	7.08	7.18	7.55	±4.4%
Relevance	8.58	8.76	8.18	8.98	8.26	8.84	7.84	7.92	8.42	±5.1%
Diversity	7.58	7.7	7.28	7.86	7.36	7.74	6.98	7.06	7.45	±4.4%
Specificity	8.14	8.32	7.78	8.54	7.88	8.58	7.4	7.48	8.02	±5.7%

The detected issues were then passed to the *self-correction* loop for repairing, ultimately producing a corrected plan tree as shown in the lower-right corner (the modified parts are highlighted in yellow for clarity).

K.3 Impact of Backbone LLMs on End-to-End Performance

Table 17 reports results for research trend analysis, which exhibit trends similar to those in Table 11.

You are an expert assistant for generating evaluation data for a paper search system.
 You are given a set of papers with structured attributes.
 Your task is to generate 20 realistic natural language queries that a human researcher might ask to find these papers.

Attributes Available for Query Construction:

Metadata

- "title": Full title of the paper.
- "authors": List of author names.
- "venue": Institutions or organizations corresponding to the authors.
- "publication_year": Year of publication.
- "venue": The conference, workshop, or journal where the paper appeared.

Aspects

- "research_topic": The broad area or domain of study (e.g., "time series similarity search", "graph neural networks").
- "problem_formulation": The precise problem formulation addressed by the paper, including how it is defined or formalized.
- "proposed_method": The main algorithm, model, or framework introduced by the paper, including its name (if available) and a brief description of its core idea or mechanism.
- "experimental_datasets": Datasets used in experiments.
- "experimental_baselines": Methods, models, or algorithms that the paper explicitly compares against in experiments (excluding ablations or internal variants).
- "experimental_results": A concise but specific summary of the reported outcomes, including metrics and improvements where available.

Query Requirements:

1. Natural phrasing: Queries must be expressed in fluent, researcher-style language (not keyword lists, Boolean operators, or artificial phrasing).
2. Aspect focus: At least 80% of queries must be centered on aspects (problem_formulation, proposed_method, datasets, baselines, results, research_topic). Metadata (title, authors, affiliations, year, venue) may be included but primarily as secondary constraints.
3. Ground truth size: Each query must correspond to at least one paper, with an ideal ground truth set of 2-10 matching papers. Do not generate queries with no valid matches or queries that would match nearly all papers.
4. Query type diversity: Queries must span three distinct types:
 - Single-attribute queries (4 queries): Target exactly one attribute. At least 3 should focus on aspects (e.g., "Which papers use the UCR dataset?", "Papers that use attention mechanisms in graph neural networks"). Metadata-only queries are rare but allowed.
 - Multi-attribute queries (10 queries): Combine multiple attributes across aspects and optionally metadata (e.g., "Find papers that introduce transformer-based models for time series forecasting on UCR datasets", "Which papers published after 2019 focus on the reverse kNN problem in trajectory search?", "Papers using learned indexes for time series similarity search that evaluate on both synthetic and real-world datasets").
 - Comparative queries (6 queries): Require relative reasoning about performance, baselines, datasets, or results. These should highlight comparisons (e.g., "Which papers outperform DTW on UCR datasets in runtime?", "Papers comparing against DeepMatcher and achieving higher F1 on Wiki dataset", "Which papers report better recall than BERT-based methods for schema matching?").
5. Attribute coverage: Collectively, the 20 queries must cover every attribute at least once across metadata and aspects.
6. No trivial queries: Avoid overly broad queries ("papers on machine learning") or impossible queries with no matching papers.
7. Realism: Queries must reflect how an actual researcher would search for papers, especially focusing on problem formulations, proposed methods, datasets, baselines, and results. Time- or venue-based filters can be added for realism but should not dominate.
8. Diversity of wording: Vary query style and phrasing (e.g., "Which papers...", "Show me papers that...", "Find papers that..."), and avoid repetitive templates.

Output Format

Return results strictly as a JSON list of objects with the following schema:

```
[
  {
    "query_id": "1",
    "query": "<natural language query>",
    "ground_truth": ["<paper id>", "<paper id>", ...],
    "attributes": ["<attribute 1>", "<attribute 2>", ...],
    "query_type": "<single-attribute factual | multi-attribute factual | in-topic comparative>"
  },
  {
    "query_id": "2",
    "query": "<natural language query>",
    "ground_truth": ["<paper id>", "<paper id>", ...],
    "attributes": ["<attribute 1>", "<attribute 2>", ...],
    "query_type": "<single-attribute factual | multi-attribute factual | in-topic comparative>"
  }
]
```

Fig. 15. Prompt template for generating factual and in-topic analytical queries.

```

You are an expert assistant for generating evaluation data for a paper search system. You are given a set of papers with
structured attributes. Your task is to generate 20 realistic natural language queries that a human researcher might ask
to find these papers.

## Attributes Available for Query Construction:
Metadata
- "title": Full title of the paper.
- "authors": List of author names.
- "affiliations": Institutions or organizations corresponding to the authors.
- "publication_year": Year of publication.
- "venue": The conference, workshop, or journal where the paper appeared.
Aspects
- "research_topic": The broad area or domain of study (e.g., "time series similarity search", "graph neural networks").
- "problem_formulation": The precise problem formulation addressed by the paper, including how it is defined or formalized.
- "proposed_method": The main algorithm, model, or framework introduced by the paper, including its name (if available) and a
  brief description of its core idea or mechanism.
- "experimental_datasets": Datasets used in experiments.
- "experimental_baselines": Methods, models, or algorithms that the paper explicitly compares against in experiments
  (excluding ablations or internal variants).
- "experimental_results": A concise but specific summary of the reported outcomes, including metrics and improvements.

## Query Requirements:
1. Cross-topic exploratory intent: All 20 queries must explore how methods, datasets, or baselines transfer or compare across
  different research topics. Queries should sound like a researcher trying to understand connections between broad areas,
  not like rigid database filters. Examples:
  - "Which papers have Transformer-based methods that have been applied to both time-series and graph data?"
  - "Show me papers where contrastive learning is used in both recommendation and retrieval tasks."
  - "Find papers that evaluate on ImageNet and also apply similar architectures to text or tabular data."
  - "Which papers apply graph neural networks across different domains?"
2. Aspect focus: Queries should primarily involve research aspects - especially proposed_method, experimental_datasets,
  experimental_baselines, and experimental_results. Metadata (authors, venue, year) may appear but should only act as
  secondary constraints.
3. Attribute combinations:
  - Most queries (~80%) should combine two broad attributes, such as:
    - proposed_method + research_topic
    - experimental_datasets + research_topic
    - experimental_baselines + problem_formulation
  - The remaining (~20%) can include three attributes for richer comparisons, e.g.:
    - proposed_method + experimental_datasets + experimental_results
  - Avoid using more than three core attributes per query.
4. Ground truth size: Each query must correspond to at least one paper, with an ideal ground truth set of 2-8 papers. Avoid
  queries that would return either no papers or nearly all papers.
5. Attribute coverage: Collectively, the 20 queries should involve all aspect attributes (research_topic, problem_formulation,
  proposed_method, experimental_datasets, experimental_baselines, experimental_results) at least once.
6. Level of specificity: Keep each query general enough to sound natural and return multiple relevant results.
  - Avoid over-constraining with unnecessary filters like year, venue, or exact numeric metrics.
  - Focus on semantic curiosity rather than structured filtering.
  - Avoid: "Find 2021 VLDB papers that outperform BERT by over 10% F1 on the WikiTables dataset."
  - Prefer: "Which methods outperform BERT across different text-related tasks?"
7. Realism: Queries must sound like authentic research questions a scholar would ask, not like artificial database filters.
  They should capture meaningful comparisons that help researchers find papers that identify methods, results, or datasets
  across different domains.
8. Diversity of wording: Use varied query phrasing styles (e.g., "Which papers...", "Show me papers that...", "Find papers
  that...") to avoid repetitive templates.

## Output Format
Return results strictly as a JSON list of objects with the following schema:
[
  {
    "query_id": "1",
    "query": "<natural language query>",
    "ground_truth": ["<paper id>", "<paper id>", ...],
    "attributes": ["<attribute 1>", "<attribute 2>", ...],
    "query_type": "cross-topic comparative"
  },
  {
    "query_id": "2",
    "query": "<natural language query>",
    "ground_truth": ["<paper id>", "<paper id>", ...],
    "attributes": ["<attribute 1>", "<attribute 2>", ...],
    "query_type": "cross-topic comparative"
  }
]
    
```

Fig. 16. Prompt template for generating cross-topic analytical queries.

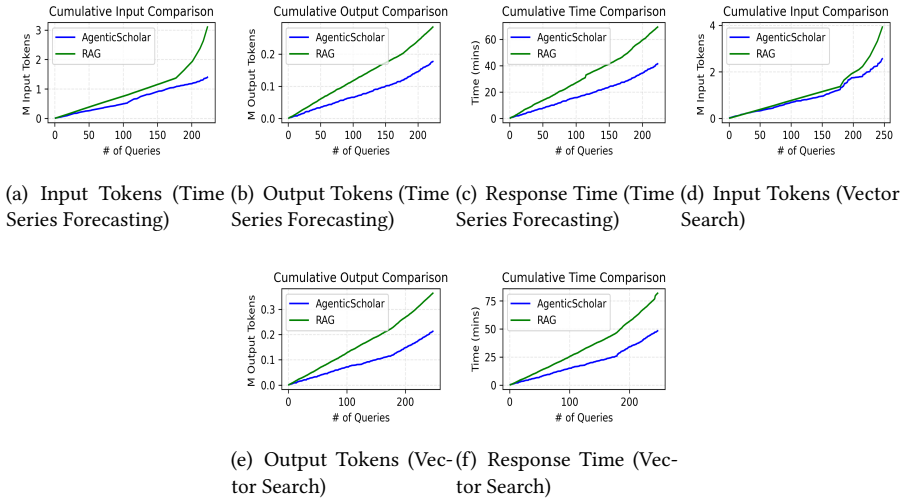


Fig. 17. Additional results of Tier-2 queries.

I'm exploring current research in computer science. Please analyze the most recent 2-3 years of literature in top conferences and identify the top 10 emerging research trends in the problems being addressed within the topic of {Input Topic}. For each trend, briefly describe what specific problem or subproblem it focuses on, why it matters (e.g., practical bottlenecks, new application needs, scalability or reliability gaps), and what is driving current progress (e.g., new environments, workloads, constraints, or user demands). Make sure the trends are distinct, specific, and evidence-based—avoid generic statements. Focus on problem formulations, tasks, and objectives rather than solution techniques.

Fig. 18. The prompt template for problem-oriented research trend analysis query

I'm exploring current research in computer science. Please analyze the most recent 2-3 years of literature in top conferences and identify the top 10 emerging research trends in the techniques employed within the topic of {Input Topic}. For each trend, briefly describe what approach, framework, or algorithmic technique it focuses on, why it matters (e.g., efficiency, interpretability, adaptability, scalability), and what is driving current progress (e.g., new models, optimization paradigms, hardware accelerators, or integration with foundation models). Make sure the trends are distinct, specific, and evidence-based—avoid generic statements. Focus on methods, architectures, and enabling techniques rather than problem formulations.

Fig. 19. The prompt template for method-oriented research trend analysis query

Please act as a creative research advisor and brainstorm 10 promising research directions stemming from the following research topic: query optimization.

For each direction, please frame it as a research question and briefly describe:

The Core Idea: The main technical or conceptual approach. The Novelty: What makes this angle new or different from existing work?

The Key Challenge: The main difficulty that would need to be overcome.

Fig. 20. The prompt template for research idea exploration query.

Please act as a creative research advisor and brainstorm 10 promising research directions stemming from the following research topic: query optimization.

For each direction, please frame it as a research question and briefly describe:

The Core Idea: The main technical or conceptual approach. The Novelty: What makes this angle new or different from existing work?

The Key Challenge: The main difficulty that would need to be overcome.

Fig. 21. The prompt template for milestone paper selection query.

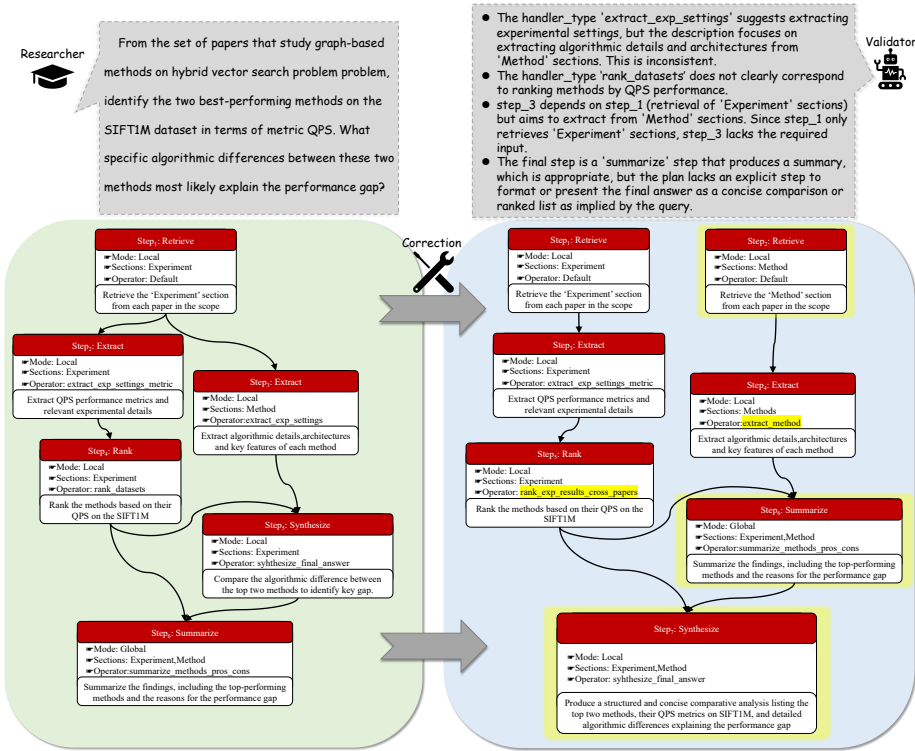
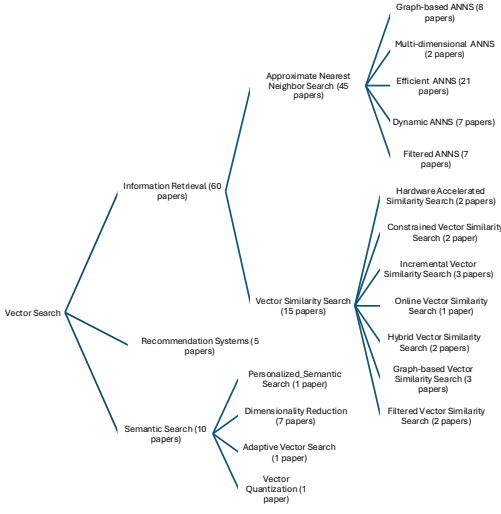
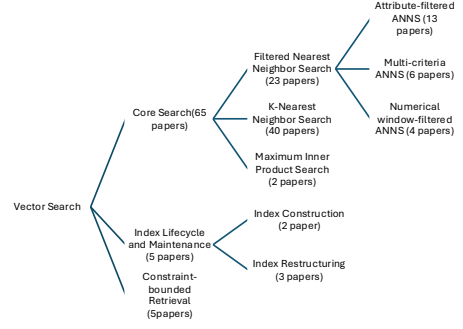
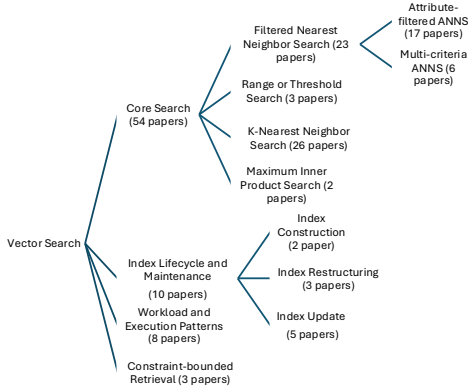
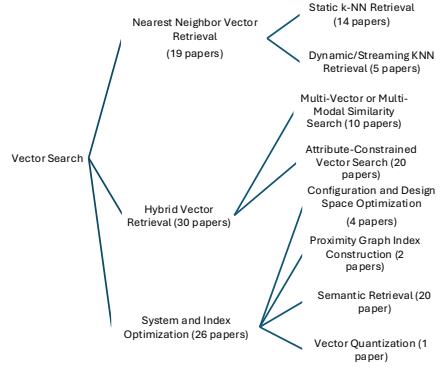


Fig. 22. Case study for self-correction.



(a) TaxoAdapt

(b) AgenticScholar_{noc}(c) AgenticScholar_{nor}

(d) AgenticScholar

Fig. 23. Problem taxonomy comparison on “Vector Search”.

Assignment Accuracy – You are checking whether one paper matches a problem-definition node.

Rules: use only fields present among name, description; treat "", " ", "n/a", "none", "null", "-" as missing; mark consistent only if title/abstract clearly targets the node's scope (use both name and description); if both title and abstract are effectively missing, mark inconsistent.

Scoring: "score" = 1 only if paper and node are consistent; else 0.

Input: Problem node { name: {{name}}, description: {{description}} }; Paper: {{paper}}.

Output (strict JSON; no prose): { "score": 0|1 }

Hierarchical Consistency – Decide if Node B BELONGS_TO Node A (problem definitions).

Rule: return { "score": 0 } if B is NOT a strict specialization/subset of A, or if this cannot be established; otherwise { "score": 1 }.

Notes: use only provided fields; treat "", " ", "n/a", "none", "null", "-" as missing; do not infer beyond broadly accepted topic knowledge.

Input: Node A: {{node_a}} ; Node B: {{node_b}}.

Output (strict JSON): { "score": 0 | 1 }

Sibling Coherence – Evaluate the sibling set under a parent problem-definition node.

Definition: children are siblings iff each fits under the parent and no child strictly subsumes/duplicates/contradicts another; granularity should be comparable.

Return 0 if ANY: parent misfit; subsumption (strict subset/superset); near-duplicate; cross-child conflict; insufficient info. Otherwise 1.

Input: Parent: {{parent_node}} ; Candidate siblings: {{sibling_nodes}}.

Output (strict JSON): { "score": 0 | 1 }

Description Accuracy – Evaluate a problem-definition node.

Objective: assess quality/consistency among present fields {description, input, output}; ignore missing ("", " ", "n/a", "none", "null", "-").

Checks: (1) clarity/specificity; (2) alignment in scope; (3) feasibility (output achievable from input when both present); (4) no contradictions; (5) no method leakage/tautology.

Scoring: if no field present, return 1 (root/placeholder); if more than 1 present and all pass and are mutually consistent, return 1; else 0.

Input: description: {{description}} ; input: {{input}} ; output: {{output}}.

Output (strict JSON): { "score": 0|1 }

Fig. 24. LLM evaluation prompts used to compute Assignment Accuracy, Hierarchical Consistency, Sibling Coherence, and Description Accuracy over problem taxonomy.

Assignment Accuracy – Check whether one paper matches a method node.
Rules: use only fields present among name, description; treat "", " ", "n/a", "none", "null", "--" as missing; mark consistent only if title/abstract clearly describes the same method/approach (or a clear variant/implementation) within the node's stated method; if both title and abstract are effectively missing, mark inconsistent.
Scoring: "score" = 1 only if the paper is consistent with the method node; else 0.
Input: Method node { name: {{name}}, description: {{description}} }; Paper: {{paper}}.
Output (strict JSON; no prose): { "score": 0 | 1 }

Hierarchical Consistency – Decide if Node B BELONGS_TO Node A.
Rule: return { "score": 0 } if B is NOT a specific subtopic/variant/implementation of A; otherwise { "score": 1 }.
Notes: use only provided fields; treat "", " ", "n/a", "none", "null", "--" as missing; do not infer beyond broadly accepted topic knowledge.
Input: Node A: {{node_a}} ; Node B: {{node_b}}.
Output (strict JSON): { "score": 0 | 1 }

Sibling Coherence – Evaluate the sibling set under a parent method node.
Definition: children are siblings iff each fits the parent's methodological family, no child strictly subsumes/duplicates/contradicts another, and all are at comparable granularity (peer variants).
Return 0 if ANY: parent misfit; subsumption (strict subset/superset); near-duplicate; core conflict with defining mechanisms/assumptions; pros/cons directly negate defining claims; insufficient information. Otherwise 1.
Input: Parent method: {{parent_node}} ; Candidate sibling methods (JSON array): {{sibling_nodes}}.
Output (strict JSON): { "score": 0 | 1 }

Description Accuracy – Evaluate a method node.
Objective: assess quality/consistency among present fields {description, pros, cons}; ignore missing ("", " ", "n/a", "none", "null", "--").
Checks: (1) clarity & specificity; (2) alignment of pros/cons with the method; (3) no contradictions (e.g., "low latency" vs "very slow"); (4) no near-duplicate bullets and consistent granularity; (5) plausibility of claims for the method class.
Scoring: if \textbf{more than one} field is present and all present fields pass and are mutually consistent, return 1; otherwise 0.
Input: description: {{description}} ; pros: {{pros}} ; cons: {{cons}}.
Output (strict JSON): { "score": 0 | 1 }

Fig. 25. LLM evaluation prompts used to compute Assignment Accuracy, Hierarchical Consistency, Sibling Coherence, and Description Accuracy over the method taxonomy.